Wind power forecasting under corrupted SCADA for edge deployment: sparse MoE with feasibility guidance

Junyu Li[a], Juntao Du[a,*]

[a]*School of Statistics and Applied Mathematics, Anhui University of Finance and Economics, Bengbu, 233030, China*

Abstract

Wind-power forecasting for farm controllers differs from benchmark forecasting because inference must run on corrupted SCADA streams under batch-1 latency and memory budgets, and forecast value only becomes meaningful after a fixed forecast-to-dispatch interface. We study that deployment-consistent setting with Ultra-LSNT, a sparse mixture-of-experts forecaster for controller-side inference, and FG-MoE, a feasibility guardrail trained from the last observed wind speed so the constraint remains available online. On Wind (CN), Ultra-LSNT remains usable under severe corruption, retaining positive skill at the Gaussian stress slice $\sigma_{\text{eff}} = 0.60$ with $R^2 = 0.7863$. FG-MoE cuts rated-power exceedance from 0.0550% to 0.0092% against a matched no-guardrail backbone while avoiding the clean/severe accuracy collapse of clip-only repair. Under a shared mapped-and-clipped dispatch-screening interface, the resulting forecast streams remain comparatively usable in cost and fallback burden relative to weaker local baselines. These results support sparse controller-side forecasting with deployable feasibility guidance as a promising controller-side comparative route under the present protocol, while leaving operator-grade validation to future work.

Keywords: Wind power forecasting, Economic dispatch, SCADA imperfections, Edge computing, Robustness, Wind power integration

1. Introduction

Wind-power forecasts enter day-ahead and intraday scheduling before they appear as benchmark scores. Operators use them in renewable offering, reserve procurement, storage coordi-

*Corresponding author.
Email addresses: ljylikezmn999@gmail.com (Junyu Li), dujuntao@aufe.edu.cn (Juntao Du)

nation, and OPF, so forecast errors change curtailment, fallback actions, and operating cost across the scheduling horizon [1–5]. In practice, farm controllers must make those forecasts from SCADA streams that can be missing, drifting, or corrupted, and they must do so under tight batch-1 latency and memory limits. The operational question is therefore whether a model remains usable after its errors pass through dispatch.

Decomposition and hybrid models reduce short-term error on non-stationary, multi-scale wind series by separating slow trends from faster fluctuations [6–8]. Attention, graph, and long-context models improve dependency modeling, horizon coupling, and transfer across related forecasting tasks [9–12]. SCADA monitoring and preprocessing studies repeatedly report missing intervals, duplicates, calibration drift, change points, and operating-region ambiguity, then recover analytical reliability through filtering, normalization, anomaly detection, and power-curve reconstruction [13–21]. Most of that evidence is still reported after telemetry has been cleaned or under server-class inference conditions.

Edge and industrial-edge studies treat batch-1 latency, memory, and communication as hard controller constraints [22–24]. Physics-guided forecasting and feasibility-aware forecasting show that turbine envelopes, power curves, and training-time constraints can steer predictions toward admissible operating regions before post-processing is needed [21, 25–27]. Dispatch-value and OPF studies show that the same forecast can lead to different cost, reserve, curtailment, and feasibility outcomes once the forecast-to-grid interface and scheduling model change [3, 4, 28–30]. A deployable forecaster therefore has to fit the device, stay physically plausible, and preserve value in the decision layer.

The literature leaves three operational gaps. First, most wind-forecasting benchmarks still evaluate cleaned inputs or server-side inference, not corrupted SCADA under controller budgets. Second, most SCADA quality-control studies clean first and forecast later, while online deployment has to absorb corrupted telemetry inside the prediction pipeline. Third, feasibility control and dispatch valuation are usually assessed in separate studies, so there is still little like-for-like evidence from corrupted input to screened operational outcome under one fixed interface.

This paper addresses those gaps on the Wind (CN) SCADA dataset. We build Ultra-LSNT for controller-side forecasting from corrupted telemetry, add FG-MoE as a deployable feasibility guardrail, and evaluate robustness, safety, hardware fit, and dispatch consequences in one pipeline. On that basis, the paper contributes four concrete pieces:

(1) Ultra-LSNT brings sparse Top-K routing and Jump Gate skipping into corrupted-SCADA forecasting under batch-1 latency and memory budgets.

(2) FG-MoE regularizes forecasts with a wind proxy that remains available online, so feasibility control does not depend on future or auxiliary states.

(3) The evaluation pipeline couples corruption injection, robustness testing, hardware profiling, and safety diagnostics under the same data and deployment assumptions.

(4) A shared mapped-and-clipped forecast-to-grid screening interface lets the same compared models be read through cost, curtailment, fallback frequency, and RTS-24 DC-screen outcomes without retuning downstream algebra per model.

2. Related work

Wind forecasting, SCADA quality control, feasibility guidance, and dispatch studies already define most pieces of the present setting. What remains rare is to evaluate them along one controller-side chain, from corrupted telemetry to dispatch-relevant outcome.

2.1. Forecasting under non-stationarity and corrupted operational data

Decomposition-based and hybrid forecasters improve short-term wind prediction by separating slowly varying trends from faster fluctuations before sequence modeling, which is especially effective on non-stationary and multi-scale wind series [6–8]. Robust decomposition frameworks then combine that signal separation with metaheuristic tuning to keep prediction stable when wind regimes shift or the series remains strongly non-stationary [8]. Hybrid attention models strengthen long- and short-range dependency capture inside the forecasting backbone, and real-time transfer-learning variants are used to correct residual drift during online prediction [9, 10]. Long-context and pretrained models such as STELLM and MMGPT4LF widen the usable receptive field and improve transfer beyond single-task wind benchmarks by reusing broader temporal and cross-task context [11, 12].

Operational SCADA studies describe much harsher input conditions. Reviews of turbine monitoring and early diagnostic work report missing channels, outliers, label ambiguity, and operating-regime changes, then rely on filtering, normalization, and channel validation before extracting reliable engineering conclusions [13–16]. Early fault-detection and normal-behavior studies already depend on identifying regime changes before maintenance or anomaly labels

become reliable [16, 18]. Change-point detection keeps normal-behavior models stable under operating shifts, deep autoencoders support system-wide anomaly detection, fleet-median filtering makes offshore failure prediction scalable, and explicit sensor-data preprocessing changes fitted power curves rather than only their residual noise [18–21].

Recent forecasting papers show how much stronger the backbone can become once the series has been decomposed or otherwise stabilized. SCADA quality-control papers show why that assumption is fragile in live telemetry and why the cleaning step cannot stay invisible in controller-side forecasting.

2.2. *Feasibility-aware and physics-guided forecasting*

Power-curve and turbine-envelope studies use cut-in, rated, and cut-out regions to flag implausible SCADA points, reconstruct consistent operating envelopes, and separate turbine behavior from sensor or regime artifacts [21, 25, 31, 32]. In practice, those constraints improve condition monitoring and materially sharpen power-curve modeling when raw telemetry contains outliers or mixed operating states [21, 25]. Performance-analysis studies use the same envelopes to distinguish turbine underperformance from sensor contamination, so feasibility information becomes part of the operating signal rather than a post-analysis label [25, 31].

Physics-guided forecasting extends the same principle into training. Physics-informed reinforcement learning has been used to keep probabilistic wind forecasts stable during extreme events, and trust-oriented work tests whether forecast explanations and constraints remain reliable enough for deployment [26, 27]. Recent reviews describe a shift from posthoc envelope checks toward training-time physical and uncertainty constraints because early regularization preserves more accuracy than repairing implausible trajectories after inference [33, 34]. The practical gain in that literature is not only fewer violations, but also a forecast distribution that stays closer to physically admissible operating regions before any repair step is applied.

Many feasibility-aware methods still depend on reconstructed variables, auxiliary states, or richer-than-online signals that may disappear at controller inference time [21, 26, 27]. The literature has made the case for training-time feasibility guidance; what remains open is how to impose it with signals that still exist online and without giving away too much usable forecast range.

2.3. Forecast-to-decision valuation and deployment constraints

Edge-computing and industrial-edge studies make the controller environment explicit by treating latency, memory, and communication as first-class design constraints for smart-grid and offshore wind applications [22–24]. In those studies, edge intelligence is the last mile only if inference budgets, resilient networking, and board-side reliability are satisfied first [22–24]. A strong forecaster that cannot satisfy those limits is not a field-ready option.

Operational-value studies ask a complementary question. Forecast uncertainty already enters unit commitment, reserve decisions, and stochastic scheduling directly [4, 28, 29]. Scenario-based economic-value studies then show that the same accuracy improvement can have very different monetary consequences across operating conditions [4]. OPF and coordinated scheduling papers show that the same forecast stream can change cost, curtailment, congestion, and feasibility differently once the network model or market mapping changes [1, 3, 30, 35]. Comparative ACOPF/DCOPF evidence pushes the same point further: network formulation itself changes how wind-forecast value is expressed downstream [30].

These studies pin down two hard constraints: a model has to meet the device budget before it can run, and its value still depends on the chosen dispatch interface. What remains scarce is a like-for-like comparison where corrupted SCADA, controller budgets, and one fixed operational mapping are all imposed at once. Table 1 summarizes representative strands around that joint setting.

Forecasting papers, SCADA diagnostics, and dispatch studies each solve part of the operational problem summarized in Table 1. Very few experiments keep corrupted input, deployment-available feasibility signals, and a fixed decision interface in the same chain. That is the setting studied here.

3. Methodology

This section defines the model inputs, sparse routing mechanism, feasibility regularizer, and evaluation criteria used in the shared protocol.

3.1. Problem setup

Given a multivariate look-back window $\mathbf{X} \in \mathbb{R}^{L \times C}$ with L historical steps and C SCADA covariates, the task is to predict an H-step active-power trajectory $\hat{\mathbf{Y}} \in \mathbb{R}^{H \times 1}$. The constrained

Table 1: Representative literature strands and their remaining gaps relative to the present controller-side setting.

Panel A

Data-side / forecasting-side strands

Study line	Typical assumption	Main strength	Most relevant missing piece
Decomposition / hybrid forecasting [6–8] controller-side forecasting	cleaned or reconstructed wind series	handles non-stationarity and multiscale variation	assumes stabilized inputs before
Attention / graph / long-context forecasting [9–12] under device-side budgets	clean benchmarks or server-class evaluation	improves representation, long dependencies, and transfer	rarely evaluated on corrupted live SCADA
SCADA anomaly / preprocessing [13–16, 18, 21] forecasting on corrupted live SCADA	raw SCADA followed by cleaning, segmentation, or filtering	identifies telemetry instability, change points, and abnormal operating regions	cleans before forecasting rather than

Panel B

Deployment-side / decision-side strands

Study line	Operational focus	Main strength	Most relevant missing piece
Edge / controller deployment [22–24] not corrupted-input forecasting	batch-1 latency, memory, and communication limits	makes field-device constraints explicit	focuses on deployment constraints,
Physics-guided / feasibility-aware forecasting [21, 25–27]	envelopes, power curves, and training-time physical discipline	adds feasibility control during or around prediction	often relies on richer-than-deployable signals
Dispatch-value / OPF studies [3, 4, 28–30] under one shared interface	forecast-to-grid interface and scheduling consequences	shows value depends on dispatch/network interface	does not compare forecasting models

Note: The table summarizes representative strands discussed in Section 2 and the gaps they leave relative to the present controller-side setting.

learning problem is

$$\min_{\theta} \ \mathbb{E}\Big[\mathcal{L}_{\text{pred}}(\hat{\mathbf{Y}}, \mathbf{Y})\Big]. \tag{1}$$

$$\hat{y}_{t+h} \in \mathcal{F}_{\text{phys}}(v_{t+h}^{\text{proxy}}), \qquad v_{t+h}^{\text{proxy}} = v_t, \quad h = 1, \ldots, H. \tag{2}$$

where $\mathcal{F}_{\text{phys}}(\cdot)$ denotes the feasible power envelope and v_{t+h}^{proxy} is the deployable wind proxy used throughout this paper. It is parameterized by turbine specifications (P_{rated}, v_{in}, v_{rated}, v_{out}). The

construction of $\mathcal{F}_{\text{phys}}$ is given in Section 3.4.

3.2. Multi-scale decomposition and embedding

The front-end separates slow trend variation from faster residual fluctuations before routing. Following decomposition–fusion practice in renewable forecasting [10, 36], we apply a moving-average operator:

$$\mathbf{X} = \mathbf{X}_{\text{trend}} + \mathbf{X}_{\text{season}}, \qquad \mathbf{X}_{\text{trend}} = \text{MA}(\mathbf{X}), \tag{3}$$

where $\text{MA}(\cdot)$ is the moving-average operator.

The residual component is embedded through parallel 1D convolutions:

$$\mathbf{E}_s = \text{Conv1D}_{k_s}(\mathbf{X}_{\text{season}}), \tag{4}$$

$$\mathbf{E}_m = \text{Conv1D}_{k_m}(\mathbf{X}_{\text{season}}), \tag{5}$$

followed by learnable fusion:

$$\mathbf{E} = \phi(\mathbf{W}_s\mathbf{E}_s + \mathbf{W}_m\mathbf{E}_m)\,, \tag{6}$$

where $\phi(\cdot)$ is a pointwise nonlinearity and $\mathbf{W}_s, \mathbf{W}_m$ are projection matrices. This decomposition-embedding block provides the input representation for sparse routing.

3.3. Sparse routing module

This module selects a small subset of experts for each token through Top-K routing. The formulation follows classical conditional computation and load-balanced gating, adapted here to time-series forecasting with a Jump Gate [37, 38].

3.3.1. Routing formulation

Given the decomposition-and-embedding front-end above, the token-wise router input is

$$\mathbf{e}_t = \mathbf{E}[t, :], \tag{7}$$

where $\mathbf{E} \in \mathbb{R}^{L\times d}$ is the fused embedding from Section 3.2 and $\mathbf{e}_t \in \mathbb{R}^d$ is the representation routed at step t.

Figure 1 shows the model interface under the main Wind (CN) setting ($L = 96$, $H = 24$): decomposition, sparse Top-K routing, Jump Gate skipping, and the feasibility-guided training path used in the reported protocol.

Frequency-aware alternatives such as VMD/EEMD can be used in the same interface [39, 40].

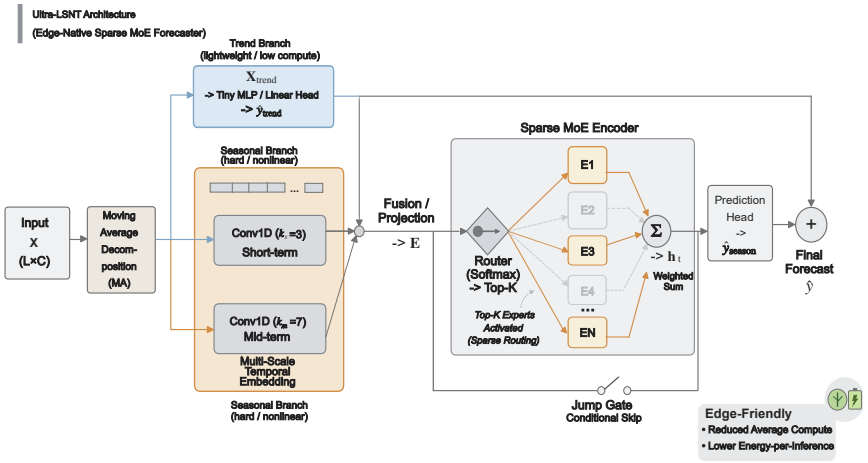


Figure 1: Ultra-LSNT architecture with decomposition, multi-scale Conv1D embedding, Top-K MoE routing, and a Jump Gate skip path together with the feasibility-guided training path used in this study.

3.3.2. *Training objective*

Let $\mathbf{e}_t \in \mathbb{R}^d$ denote the embedding at step t. Let $\{E_i(\cdot)\}_{i=1}^N$ denote expert networks:

$$E_i(\mathbf{e}_t) = \sigma(\mathbf{W}_i \mathbf{e}_t + \mathbf{b}_i), \tag{8}$$

where $\mathbf{W}_i \in \mathbb{R}^{d\times d}$ and $\mathbf{b}_i \in \mathbb{R}^d$.

A router computes logits and probabilities:

$$\mathbf{z}_t = \mathbf{W}_r \mathbf{e}_t + \boldsymbol{\epsilon}_t, \qquad \mathbf{p}_t = \mathrm{Softmax}(\mathbf{z}_t), \tag{9}$$

where $\mathbf{W}_r \in \mathbb{R}^{N\times d}$ and $\boldsymbol{\epsilon}_t \sim \mathcal{N}(0, \eta_e^2 \mathbf{I})$. The exploration scale is annealed by epoch index e:

$$\eta_e = \eta_0 \exp(-e/\tau_\eta). \tag{10}$$

Top-K routing selects $\mathcal{S}_t = \mathrm{TopK}(\mathbf{p}_t, K)$ and aggregates expert outputs as

$$\mathbf{o}_t = \sum_{i\in\mathcal{S}_t} p_{t,i}\, E_i(\mathbf{e}_t). \tag{11}$$

Training uses prediction loss and routing regularization.

Point forecasting is optimized by MSE:

$$\mathcal{L}_{\text{pred}} = \frac{1}{H} \sum_{h=1}^{H} \|\hat{y}_{t+h} - y_{t+h}\|_2^2 . \tag{12}$$

Router entropy and expert disagreement are defined as

$$H_t = -\sum_{i=1}^{N} p_{t,i} \log(p_{t,i} + \epsilon), \tag{13}$$

with $\epsilon = 10^{-8}$, and

$$D_t = \frac{1}{|\mathcal{S}_t|} \sum_{i \in \mathcal{S}_t} \left(\hat{y}^{(i)}_{t+h} - \bar{y}_{t+h} \right)^2. \tag{14}$$

$$\bar{y}_{t+h} = \frac{1}{|\mathcal{S}_t|} \sum_{i \in \mathcal{S}_t} \hat{y}^{(i)}_{t+h}. \tag{15}$$

These quantities parameterize an adaptive reserve rule:

$$\sigma_{w,t} = \min\left(\sigma_{\max}, \sigma_{\text{base}} + \alpha_H H_t + \alpha_D D_t \right), \tag{16}$$

where σ_{base}, α_H, α_D, and $\sigma_{\max}$ are fixed coefficients. Optional probabilistic heads are

$$\mathcal{L}_{\text{pin}} = \frac{1}{|\mathcal{Q}|H} \sum_{q \in \mathcal{Q}} \sum_{h=1}^{H} \rho_q\left(y_{t+h} - \hat{y}^{(q)}_{t+h} \right). \tag{17}$$

$$\rho_q(e) = \max(qe, (q-1)e). \tag{18}$$

$$\mathcal{L}_{\text{nll}} = \frac{1}{H} \sum_{h=1}^{H} \left(\frac{(y_{t+h} - \mu_{t+h})^2}{2\sigma^2_{t+h}} + \log \sigma_{t+h} \right). \tag{19}$$

The router load-balancing regularizer is

$$\mathcal{L}_{\text{aux}} = \sum_{i=1}^{N} \left(\bar{p}_i - \frac{1}{N} \right)^2, \qquad \bar{p}_i = \frac{1}{T} \sum_{t=1}^{T} p_{t,i}, \tag{20}$$

and the routing-module objective is

$$\mathcal{L}_{\text{base}} = \mathcal{L}_{\text{pred}} + \lambda_r \mathcal{L}_{\text{aux}}. \tag{21}$$

3.4. Feasibility-guided envelope regularization

FG-MoE adds a penalty that discourages forecasts outside the turbine envelope. The envelope is built from aerodynamic power conversion and a SCADA-observable proxy that remains available at deployment.

A horizontal-axis turbine is bounded by

$$P_{\text{aero}}(v) = \frac{1}{2} \rho A v^3 C_p(\lambda_{\text{tsr}}, \beta), \qquad A = \pi R^2, \tag{22}$$

where ρ is air density, A is rotor swept area, v is hub-height wind speed, and C_p is the power coefficient with $0 \le C_p \le C_{p,\max} \le 16/27$. The operational envelope is parameterized by $(P_{\text{rated}}, v_{\text{in}}, v_{\text{rated}}, v_{\text{out}})$ following standard wind-turbine power-curve conventions and test practice [21, 31, 32].

Because future wind-speed states are unavailable during deployment, FG-MoE uses the last observed hub-height wind speed and repeats it across the prediction horizon:

$$v_{t+h}^{\text{proxy}} = v_t, \qquad h = 1, \dots, H. \tag{23}$$

This choice matches the deployable proxy used in both training and evaluation, and avoids depending on unavailable future wind observations.

Using v_{t+h}^{proxy}, the envelope is

$$P_{\text{env}}(v^{\text{proxy}}) = \begin{cases} 0, & v^{\text{proxy}} < v_{\text{in}}, \\ P_{\text{rated}} \left(\frac{v^{\text{proxy}} - v_{\text{in}}}{v_{\text{rated}} - v_{\text{in}}} \right)^3, & v_{\text{in}} \le v^{\text{proxy}} < v_{\text{rated}}, \\ P_{\text{rated}}, & v_{\text{rated}} \le v^{\text{proxy}} \le v_{\text{out}}, \\ 0, & v^{\text{proxy}} > v_{\text{out}}. \end{cases} \tag{24}$$

Let $\tilde{y}_{t+h}^{\text{raw}} = \hat{y}_{t+h}/P_{\text{rated}}$, $\tilde{P}_{\text{env}}(v_{t+h}^{\text{proxy}}) = P_{\text{env}}(v_{t+h}^{\text{proxy}})/P_{\text{rated}}$, and let $\mathbb{I}_{\text{cut}}(v_{t+h}^{\text{proxy}}) = 1$ when $v_{t+h}^{\text{proxy}} < v_{\text{in}}$ or $v_{t+h}^{\text{proxy}} > v_{\text{out}}$. FG-MoE regularizes the raw forecast through

$$\begin{aligned} \mathcal{L}_{\text{physics}} = \frac{1}{H} \sum_{h=1}^{H} \Big[& \max(0, -\tilde{y}_{t+h}^{\text{raw}})^2 + \max(0, \tilde{y}_{t+h}^{\text{raw}} - 1)^2 \\ & + \max(0, \tilde{y}_{t+h}^{\text{raw}} - \tilde{P}_{\text{env}}(v_{t+h}^{\text{proxy}}))^2 \\ & + \mathbb{I}_{\text{cut}}(v_{t+h}^{\text{proxy}})(\tilde{y}_{t+h}^{\text{raw}})^2 \Big]. \end{aligned} \tag{25}$$

For the posthoc clip-only control and the fixed dispatch screening interface, we additionally use the deterministic projection

$$\begin{aligned} \tilde{y}_{t+h}^{\text{clip}} = \text{clip}\Big(& \tilde{y}_{t+h}^{\text{raw}}, 0, \\ & \tilde{P}_{\text{env}}(v_{t+h}^{\text{proxy}}) \Big), \end{aligned} \tag{26}$$

which is not the core FG-MoE training path in the main protocol. Combining Eqs. (12), (20), and (25), the final objective is

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{pred}}(\hat{\mathbf{Y}}^{\text{raw}}, \mathbf{Y}) + \lambda_r \mathcal{L}_{\text{aux}} + \lambda_2 \mathcal{L}_{\text{physics}}. \tag{27}$$

Algorithm 1 Ultra-LSNT training workflow with SCADA imperfection injection.

Input: training samples $(\mathbf{X}, \mathbf{Y})$ with $\mathbf{X} \in \mathbb{R}^{L \times C}$, $\mathbf{Y} \in \mathbb{R}^{H \times 1}$

Hyperparameters: experts N, Top-K, router regularizer λ_r, physics weight λ_2, noise config σ_{cfg}, industrial scale α, Jump Gate threshold τ

$\sigma_{\text{eff}} \leftarrow \alpha \cdot \sigma_{\text{cfg}}$

$\tilde{\mathbf{X}} \leftarrow \text{InjectImperfections}(\mathbf{X}; \sigma_{\text{cfg}}, \sigma_{\text{eff}})$

(Gaussian noise, drift, quantization; see Sec. 4.2 for definitions.)

$(\mathbf{X}_{\text{trend}}, \mathbf{X}_{\text{resid}}) \leftarrow \text{Decompose}(\tilde{\mathbf{X}})$

$\mathbf{Z} \leftarrow \text{MultiScaleEmbed}([\mathbf{X}_{\text{trend}}, \mathbf{X}_{\text{resid}}])$

for $b = 1$ **to** N_{blk} **do**

  $\mathbf{g} \leftarrow \text{Router}_b(\mathbf{Z})$ *(routing probabilities)*

  $\mathcal{E} \leftarrow \text{TopK}(\mathbf{g}, K)$ *(selected experts)*

  $\mathbf{Z} \leftarrow \text{AggregateExperts}_b(\mathbf{Z}, \mathcal{E})$

  $\mathbf{Z} \leftarrow \mathbf{Z} + \text{TrendInteract}_b(\mathbf{Z}, \mathbf{X}_{\text{trend}})$

end for

$\hat{\mathbf{Y}}^{\text{raw}} \leftarrow \text{ForecastHead}(\mathbf{Z})$

$\hat{\mathbf{V}}^{\text{proxy}} \leftarrow \text{RepeatLastWind}(\tilde{\mathbf{X}})$

$\mathcal{L}_{\text{pred}} \leftarrow \text{MSE}(\hat{\mathbf{Y}}^{\text{raw}}, \mathbf{Y})$

$\mathcal{L}_{\text{aux}} \leftarrow \text{LoadBalance}(\mathbf{g})$

$\mathcal{L}_{\text{physics}} \leftarrow \text{PhysicsPenalty}(\hat{\mathbf{Y}}^{\text{raw}}, \hat{\mathbf{V}}^{\text{proxy}})$

Update parameters to minimize $\mathcal{L}_{\text{pred}} + \lambda_r \mathcal{L}_{\text{aux}} + \lambda_2 \mathcal{L}_{\text{physics}}$

For reproducibility, Algorithm 1 summarizes training with imperfection injection and feasibility guidance, Algorithm 2 summarizes edge inference with Jump Gate skipping, and Fig. 2 shows the workflow from SCADA measurements to dispatch screening outcomes.

Figure 2 shows the end-to-end dispatch-screening chain. The shared scale mapping and deterministic clipping belong to the screening interface, not to the FG-MoE training path used for the accuracy and safety results.

Algorithm 2 Edge inference workflow with Jump Gate conditional skipping.

Input: window $\mathbf{X} \in \mathbb{R}^{L \times C}$
$(\mathbf{X}_{\text{trend}}, \mathbf{X}_{\text{resid}}) \leftarrow \text{Decompose}(\mathbf{X})$
$\mathbf{Z} \leftarrow \text{MultiScaleEmbed}([\mathbf{X}_{\text{trend}}, \mathbf{X}_{\text{resid}}])$
for $b = 1$ **to** N_{blk} **do**
  if $\text{JumpGate}_b(\mathbf{Z}; \tau) = \text{SKIP}$ **then**
    continue
  end if
  Route experts and aggregate as in Algorithm 1
end for
$\hat{\mathbf{V}}^{\text{proxy}} \leftarrow \text{RepeatLastWind}(\mathbf{X})$
$\hat{\mathbf{Y}}^{\text{raw}} \leftarrow \text{ForecastHead}(\mathbf{Z})$
Optional screening: $\hat{\mathbf{Y}}^{\text{clip}} \leftarrow \text{PhysicsClip}(\hat{\mathbf{Y}}^{\text{raw}}, \hat{\mathbf{V}}^{\text{proxy}})$
Output: $\hat{\mathbf{Y}}^{\text{raw}}$ *(or* $\hat{\mathbf{Y}}^{\text{clip}}$ *for the clip-only control / dispatch interface)*

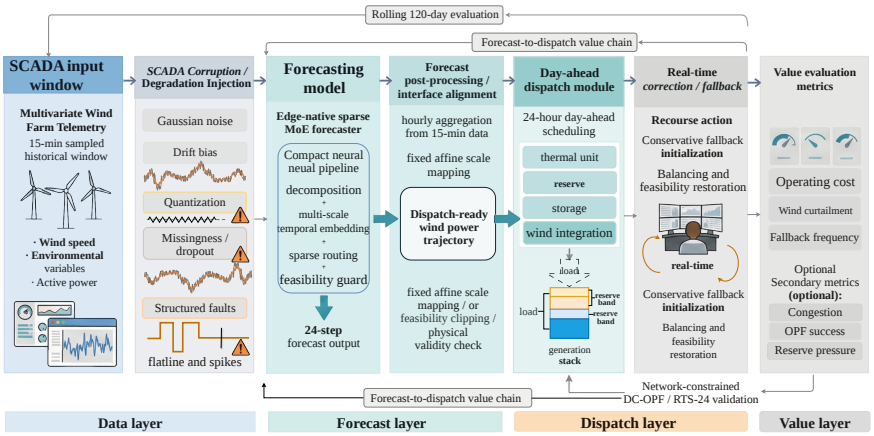


Figure 2: Fixed-interface dispatch screening pipeline from SCADA window to dispatch outcomes.

3.5. *Conditional skipping for edge inference*

The Jump Gate defines token-wise conditional skipping. The gate score is

$$I_t = \text{Sigmoid}(\mathbf{w}_g^\top \mathbf{e}_t), \tag{28}$$

with hard-threshold transition

$$\mathbf{o}_t = \begin{cases} \mathbf{o}_{t-1}, & I_t < \tau, \\ \mathrm{MoE}(\mathbf{e}_t), & \text{otherwise}, \end{cases} \tag{29}$$

where τ is the threshold parameter.

During training, the hard gate is relaxed as

$$\tilde{z}_t = \sigma((I_t - \tau)/s), \qquad \mathbf{o}_t = \tilde{z}_t \cdot \mathrm{MoE}(\mathbf{e}_t) + (1 - \tilde{z}_t) \cdot \mathbf{o}_{t-1},$$

where $s > 0$ is temperature.

Ultra-LSNT-Lite is defined by reducing sparse-module capacity with Top-1 routing:

$$(N, K, \#\text{MoE blocks}, d) \;\rightarrow\; (2, 1, 1, 96), \tag{30}$$

where d is the hidden dimension.

Complexity follows directly. Let d denote hidden dimension. A dense N-expert computation scales as $O(N \cdot L \cdot d^2)$, whereas Top-K routing scales as $O(K \cdot L \cdot d^2)$ with $K \ll N$. If Jump Gate is enabled, the effective token count is multiplied by the non-skip ratio.

3.6. Evaluation metrics and robustness criteria

Primary point-forecasting metrics are R^2, RMSE, and MAE, computed after inverse scaling in the original physical units of each dataset and aggregated over the full horizon and test split. We retain MAPE for completeness, but because wind-power series contain frequent near-zero periods, ratio-based errors can be disproportionately inflated even with numerical safeguards; MAPE is therefore treated as a supplementary diagnostic rather than a headline metric.

$$R^2 = 1 - \frac{\sum_{t=1}^{T}(y_t - \hat{y}_t)^2}{\sum_{t=1}^{T}(y_t - \bar{y})^2}. \tag{31}$$

$$\mathrm{RMSE} = \sqrt{\frac{1}{T}\sum_{t=1}^{T}(y_t - \hat{y}_t)^2}. \tag{32}$$

$$\mathrm{MAE} = \frac{1}{T}\sum_{t=1}^{T}|y_t - \hat{y}_t|. \tag{33}$$

$$\mathrm{MAPE} = \frac{100}{T}\sum_{t=1}^{T}\left|\frac{y_t - \hat{y}_t}{y_t + \epsilon}\right|. \tag{34}$$

Here $\bar{y}$ is the sample mean of y_t, and ϵ is a small constant used to avoid numerical instability when y_t approaches zero.

Average point errors do not fully characterize controller-side usefulness. We therefore audit tail-error, ramp, smoothness, and routing-stability indicators alongside the primary metrics, and for probabilistic variants we additionally report PICP, PINAW, and CRPS. Robustness tests combine continuous perturbations (Gaussian noise, drift, quantization) with structured SCADA faults (flatline, spikes, correlated dropouts). Throughout the main protocol, $\sigma_{\text{eff}} = 0.15$ is treated as the empirical upper-band anchor, while $\sigma_{\text{eff}} \in \{0.30, 0.45, 0.60\}$ defines an explicit stress envelope.

4. Experimental design and comparison setup

The experiments use one shared pipeline for data preparation, benchmark selection, corruption stress, and dispatch-coupled screening.

4.1. Data, preprocessing, and tasks

The primary case study is ultra-short-term wind-power forecasting using SCADA measurements from a utility-scale wind farm in Xinjiang, China. The setup is intended to reflect operational conditions with sensor uncertainty and non-stationary weather. Inputs include wind and environmental variables, and the target is aggregated active power.

Table 2: Dataset characteristics and SCADA degradation protocol.

Item	Specification
Panel A	
Multi-domain dataset spatiotemporal characteristics	
Wind (CN) SCADA (primary)	15-min sampling, 121,528 records, train/test chronology 80%/20% (97,222/24,306), train range 2021-04-02 00:00 to 2022-03-12 15:45, test range 2022-03-12 16:00 to 2022-06-30 23:45, 8 input channels, target active power in kW.
Wind (US) transfer	5-min sampling, 105,120 records, full-year range 2012-01-01 00:00 to 2012-12-31 23:55, 10 input channels (time/covariates) with wind-power target in MW.
Air Quality transfer	Daily city-panel records, 588,644 samples after preprocessing, range 2015-01-01 to 2023-05-31, 7 pollutant/air-quality channels (AQI, PM2.5, PM10, SO_2, NO_2, CO, O_3).
GEFCom Load transfer	1-hour sampling, 38,070 records, range 2004-01-01 00:00:00 to 2008-06-30 05:00:00, univariate load target with calendar-aligned chronological split.

Table 2: Dataset characteristics and SCADA degradation protocol (continued).

Item	Specification
Unified task protocol	All datasets use chronological splitting, train-fitted z-score normalization, inverse scaling before metric reporting, and common QC (duplicate removal, missing-value handling, NaN/Inf sanity checks). Main forecasting setup uses lookback/horizon $(L, H) = (96, 24)$.
Panel B	
SCADA degradation and stress protocol (Wind CN main benchmark)	
Gaussian noise (inference-time)	Configured severity $\sigma_{\text{cfg}} \in \{0.00, 0.10, 0.20, 0.30, 0.40\}$ with industrial scaling $\sigma_{\text{eff}} = \alpha\sigma_{\text{cfg}}$, $\alpha = 1.5$, giving effective tiers $\sigma_{\text{eff}} \in \{0.00, 0.15, 0.30, 0.45, 0.60\}$.
Drift bias (random-walk)	Persistent bias injected as a channel-wise random walk with step scale $\sigma_{\text{eff}} \cdot \text{std}(x)$; this emulates calibration drift and low-frequency sensor offset accumulation.
Quantization noise	Deterministic rounding with step size $\Delta = \sigma_{\text{eff}} \cdot \text{std}(x)$, i.e., $x_t^{\text{quant}} = \Delta \cdot \text{round}(x_t/\Delta)$, to emulate low-precision SCADA/edge telemetry.
Structured SCADA faults	Flatline, spike/heavy-tail outlier, and correlated dropout with contiguous windows; severity levels $\{0.2, 0.6\}$, with level 0.6 used as the severe-certification slice in main robustness tables.
Missingness and interpolation	i.i.d. masking rate $p \in \{0.1, 0.3\}$ with paired `ffill`/linear interpolation (3 seeds, shared masks across models); burst/correlated missingness is represented by structured dropout faults.
Empirical severity anchor	Channel-wise diff-proxy calibration on Wind (CN) yields median 0.045, P90 0.091, max 0.121; therefore $\sigma_{\text{eff}} = 0.15$ is treated as empirical upper-band neighborhood, while $\{0.30, 0.45, 0.60\}$ are explicit stress envelopes.
Dispatch coupling protocol	For dispatch, 15-min Wind (CN) is aggregated to 1-hour resolution; 120-day rolling copper-plate and IEEE RTS-24 DC-OPF screening are both reported under fixed affine scale mapping and fixed feasibility clipping rules.

Table 2 gathers dataset scope, stress tiers ($\sigma_{\text{eff}} \in \{0.15, 0.30, 0.45, 0.60\}$), and dispatch aggregation (15-min to hourly) in one place, so the main comparisons do not shift preprocessing or severity settings from model to model. Figure 3 shows that the study covers heterogeneous temporal patterns rather than a single profile. To test transfer behavior, we include three additional tasks: *Wind (US)*, *Air Quality*, and *GEFCom Load*. All datasets are split chronologically to prevent leakage. Z-score normalization is fitted on the training split only and inverse scaling is applied before reporting physical-unit metrics. The same quality-control pipeline (duplicate removal, missing-value handling, NaN/Inf checks) is applied across all experiments. These transfer results are retained as secondary boundary-setting diagnostics rather than as extensions of the main controller-side claim.

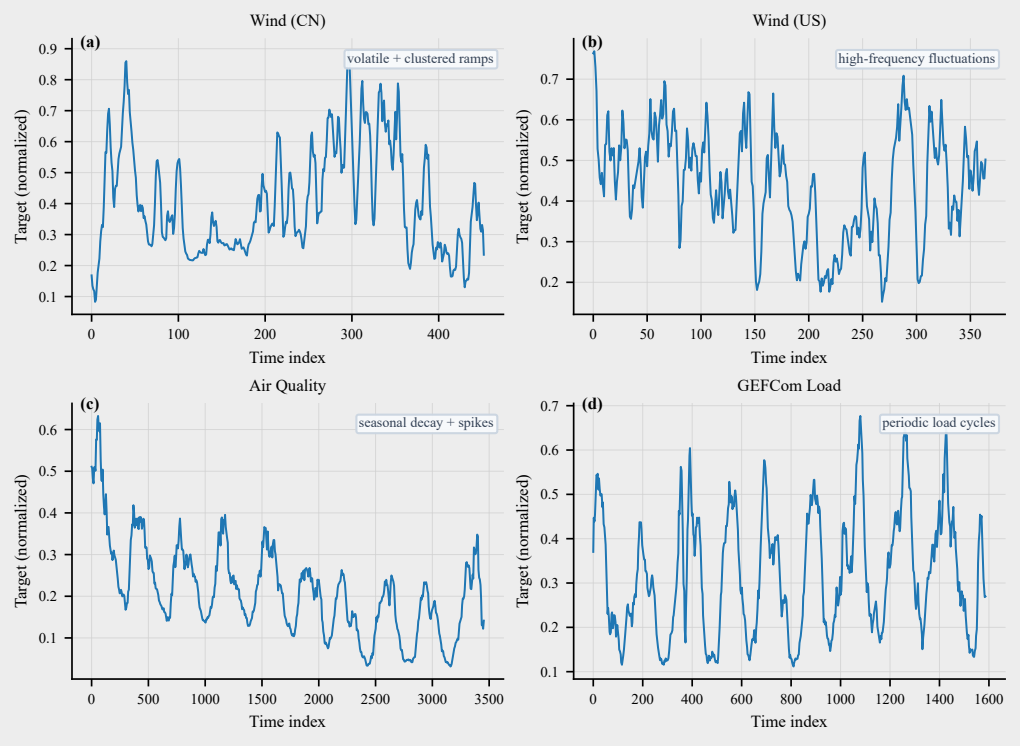


Figure 3: Representative raw-series excerpts for Wind (CN), Wind (US), Air Quality, and GEFCom Load. Series are min-max normalized within each dataset for visual comparability and plotted with dataset-specific excerpt length.

4.2. Baselines and comparison logic

The main benchmark separates two questions: how models rank under the same corrupted-input stress slices, and how much fault augmentation changes each architecture when the backbone is held fixed. Chronos and Moirai remain context references for extended-context behavior, while LightGBM and Persistence anchor feature-engineered and naive-history baselines; long-tail hybrids and constrained-budget metaheuristics move to the supplementary tables in Appendix G so the main comparison stays focused.

LightGBM and Persistence. Both use the same chronological split, history/forecast horizon $(L, H) = (96, 24)$, train-fitted z-score normalization, inverse scaling, and test-time corruption schedule as the deep models. LightGBM uses a multi-output `LGBMRegressor` on flattened history only. Its hyperparameters are fixed in advance (`n_estimators`=500, `learning_rate`=0.05, `random_state`=42, `n_jobs`=-1), so it remains a transparent tabular

baseline rather than a separately tuned search exercise. Persistence repeats the last observed target across the forecast horizon.

Controller-side usefulness also depends on tail error, ramp behavior, smoothness, and routing stability; for probabilistic variants we additionally report PICP, PINAW, and CRPS. Latency, parameter count, model size, and an analytical energy proxy complete the same picture, because a useful model must remain robust under corrupted SCADA and still fit batch-1 deployment constraints. Exact perturbation definitions are deferred to Appendix D [41, 42].

4.3. Dispatch interface and screening setup

The dispatch analysis fixes one forecast-to-grid mapping and one dispatch rule so changes in cost, curtailment, fallback frequency, and DC-OPF outcomes can be attributed to the forecast stream rather than to different downstream algebra. Broader mapping-sensitivity checks are moved to the supplementary material. Dispatch results in this paper should be interpreted as fixed-interface comparative screening rather than evidence that any raw model output is directly dispatch-ready.

The operational model is a 24-hour day-ahead problem with 1-hour resolution, one thermal unit, upward reserve procurement, and storage [4, 43, 44]. Forecast impact is summarized through operating cost, wind curtailment, and feasibility-restoration (fallback) frequency. We also run a two-stage DA/RT DC-OPF supplement on IEEE RTS-24; it captures coarse active-power network effects under linearized physics and is not an AC-feasibility test [30].

The dispatch grid is hourly. We therefore aggregate Wind (CN) from 15-min to 1-hour resolution by averaging each four consecutive records (Table 2). Each forecaster then provides a 24-step daily vector on that hourly grid, and we evaluate Ultra-LSNT, iTransformer, TimeMixer, and DLinear over a 120-day rolling window. A day is tagged as `fallback_x0` when the optimizer can close only with conservative fallback initialization under unchanged algebraic constraints.

Raw forecasts are reported once to show that the dispatch problem does not close directly: in the raw-scale stress check, all models remain fallback-only over the full 120-day window. Comparative dispatch metrics are therefore reported after one archived global mapping and deterministic clipping rule are applied to every model output. The interface is fixed across models rather than tuned per model. Accordingly, Table 5 and Figure 5 do not evaluate raw forecasts as deployable dispatch inputs; they evaluate model-separated forecast streams after one shared archived mapping and deterministic clipping rule.

To keep operating bounds consistent across models in rolling runs, we apply train-split bound clipping to each daily forecast vector:

$$\tilde{w}_t = \mathrm{clip}(s\hat{w}_t, 0, w_{\max}), \tag{35}$$

$$s = \min\left(1, \frac{w_{\max}}{\max_t \hat{w}_t + \epsilon}\right), \tag{36}$$

where $w_{\max} = \max(w_{\text{train}})$ is computed on the hourly training split and ϵ is a small constant. Results under this mapping are reported in Table 5, and both raw/mapped rolling summaries are included in supplementary evidence.

All monetary coefficients are in CNY. Because absolute totals depend on price coefficients and reference year, we report both absolute values and relative reductions against fixed baselines. With $\Delta t = 1\,\mathrm{h}$, summed kW schedules correspond to kWh energy over the horizon. Dispatch metrics are always computed against realized wind output on the same test day for all models.

We include one UQ-aware reserve variant where $\sigma_{w,t}$ scales with routing entropy and expert disagreement from the MoE forward pass. For comparability, we also report the fixed-proportional reserve rule $\sigma_{w,t} = 0.2|\hat{w}_t|$.

Note on network constraints. The copper-plate model isolates how forecast errors propagate to reserve, curtailment, and operating cost, while RTS-24 adds a comparative DC network check. DC-feasible schedules are not guaranteed to be AC-feasible.

All experiments run in one pipeline with fixed preprocessing, chronological splitting, and consistent metric computation. Full environment/training settings are reported in Appendix C. Source code is released at `https://github.com/b1ue13e/Ultra-LSNT` (MIT), and Wind (CN) data plus preprocessing/evaluation scripts are released for end-to-end reproduction.

5. Results

The evidence is organized around four questions: how much skill survives corrupted SCADA, what feasibility control costs or saves, how much of the forecast signal reaches dispatch under the fixed mapping, and which operating points remain attractive in (K, τ, λ_2) space.

5.1. Robustness under SCADA corruption

The main robustness finding is not that Ultra-LSNT dominates the mixed reference panel, but that it preserves a positive corrupted-input baseline under the controller-side protocol where sim-

pler local baselines can collapse. At the severe Gaussian slice, the long-context reference models remain highest ($R^2 = 0.8875$ for Moirai-2.0-R-Small and 0.8792 for Chronos-Bolt-Small), iTransformer and Persistence follow (0.8717 and 0.8612), Ultra-LSNT stays positive at 0.7863, and DLinear falls to -0.9903. The practical distinction is that the controller-side model preserves a positive corrupted-input baseline where a simpler local baseline collapses; the reference rows provide context under the same corruption protocol but are not edge-feasible deployment peers.

Matched-control evidence sharpens that interpretation. When architecture is held fixed and only the training condition changes, fault augmentation improves the severe-slice mean R^2 from 0.2402 to 0.3471 for DLinear, 0.7452 to 0.8417 for TimeMixer, 0.8478 to 0.8580 for iTransformer, and 0.8135 to 0.8180 for Ultra-LSNT. The largest gains appear in the weaker baselines, while Ultra-LSNT moves only modestly because it already starts from a more stable corrupted-input baseline.

At the empirical anchor $\sigma_{\text{eff}} = 0.15$, archived intermediate-slice evidence for the six local context baselines remains in positive-skill territory (DLinear 0.7814, LightGBM 0.9340, Persistence 0.9140, Ultra-LSNT 0.8687, TimeMixer 0.8847, iTransformer 0.8993), matching the calibration reported in Appendix D (median 0.045, P90 0.091, max 0.121). The main tables therefore use $\sigma_{\text{eff}} = 0.15$ as the empirical anchor and $\sigma_{\text{eff}} \in \{0.30, 0.45, 0.60\}$ as explicit stress levels.

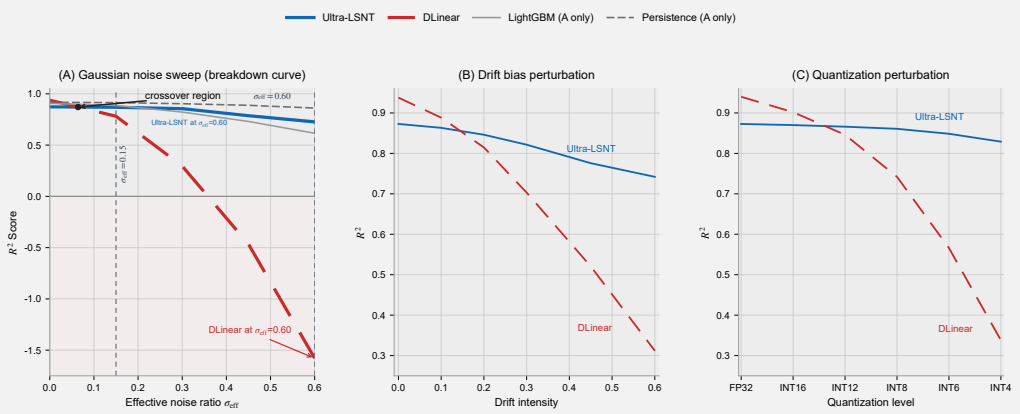


Figure 4: Robustness summary under Gaussian noise, drift bias, and quantization perturbations on Wind (CN). Panel A reports the full Gaussian sweep ($\sigma_{\text{eff}} \in \{0.00, 0.15, 0.30, 0.45, 0.60\}$) for Ultra-LSNT, DLinear, LightGBM, and Persistence, with $\sigma_{\text{eff}} = 0.15$ as the empirical upper-band anchor and $\sigma_{\text{eff}} = 0.60$ as the severe stress-envelope anchor. Panels B–C report drift and quantization trajectories for Ultra-LSNT and DLinear under the same protocol.

Table 3: Audited robustness evidence on Wind (CN).

Panel A

Clean-trained context panel (8 models; edge-feasible models + reference models)

Model	Clean R^2	Clean RMSE	R^2@0.60	RMSE@0.60	Rel. RMSE inc. (%)
DLinear	0.9494	7406.87	-0.9903	46449.77	527.12
TimeMixer	0.8886	8498.38	0.8337	10382.59	22.17
iTransformer	0.9012	8004.05	0.8717	9117.01	13.90
Ultra-LSNT	0.9346	8419.91	0.7863	15218.84	80.75
LightGBM	0.9563	6880.91	0.7121	17665.50	156.73
Persistence	0.9175	7313.94	0.8612	9485.55	29.69
Chronos-Bolt-Small	0.9644	6216.19	0.8792	11443.18	84.09
Moirai-2.0-R-Small	0.9686	5834.04	0.8875	11045.80	89.33

Panel B

Matched-control severe-slice panel

Model	Clean-trained R^2@0.60	Fault-augmented R^2@0.60	Gain
DLinear	0.2402	0.3471	+0.1069
TimeMixer	0.7452	0.8417	+0.0965
iTransformer	0.8478	0.8580	+0.0102
Ultra-LSNT	0.8135	0.8180	+0.0045

Note: Panel A provides an eight-model context view at clean and severe Gaussian slices. RMSE is retained in the main table and MAE is moved to Appendix G.2 for completeness. The realistic operating anchor is $\sigma_{\text{eff}} = 0.15$, whereas $\sigma_{\text{eff}} \in \{0.30, 0.45, 0.60\}$ defines the stress envelope. Chronos and Moirai are reported as long-context reference models under zero-shot extended-context usage (512-step input context, same 24-step target window and evaluation split) rather than as edge-feasible controller-side models. Detailed audit-file manifests are provided in the supplementary evidence bundle.

Figure 4 extends the same comparison across Gaussian, drift, and quantization trajectories. The full-σ sweep reinforces the same point as Table 3: corrupted-operation robustness should be read from the degradation path, not inferred from the clean slice alone.

5.2. Hardware and safety trade-offs

FG-MoE is valuable because it reduces infeasible outputs without paying the accuracy penalty of clip-only repair. Under the matched backbone in Table 4, clip-only drives rated-power exceedance to zero but pushes clean and severe-slice R^2 down to 0.4319 and 0.3120, whereas FG-MoE keeps those values at 0.8747 and 0.8286 while lowering exceedance to 0.0092%. The

Table 4: Hardware profile and safety diagnostics under protocol P1.

Panel A

Batch-1 edge profile (CPU/ARM)

Model	CPU Latency (ms)	ARM Latency (ms)	CPU Throughput (Hz)	Memory (MiB)
Ultra-LSNT (full)	3.714	8.795	269.3	22.2830
Ultra-LSNT-Lite (opt.)	1.171	5.007	854.0	1.3327
DLinear	0.265	0.102	3767.6	0.0178
Mamba (logged) (bf)	0.861	0.861	1161.1	0.1468
VMD-PSO-LSTM (bf)	10.297	10.297	97.1	0.2468
CEEMDAN-WOA-GRU (bf)	11.082	11.082	90.2	0.2130
ARIMA (logged) (bf)	2.183	2.183	458.0	0.0000
SVR (logged) (bf)	2.750	2.750	363.6	0.5410
SSA-ELM (bf)	0.024	0.024	42409.6	0.8107

Panel B

Guardrail operating-point comparison

Setting	Clean R^2	R^2@0.60	Rated-power exceedance (%)
Ultra-LSNT (no guardrail)	0.8727	0.8273	0.0550
Ultra-LSNT + clip-only (posthoc)	0.4319	0.3120	0.0000
FG-MoE (full-trained)	0.8747	0.8286	0.0092

Note: Panel A reports direct batch-1 measurements for Ultra-LSNT, Ultra-LSNT-Lite, and DLinear, while the remaining `logged/bf` rows are archived or backfilled screening references rather than same-hardware, same-implementation head-to-head measurements. Panel B reports the protocol-P1 comparison among no guardrail, posthoc clip-only repair, and FG-MoE. Additional cut-region and envelope-deviation diagnostics are provided in the supplementary material.

result supports recent physics-guided forecasting arguments that safety control is most effective when it reshapes the learned forecast distribution during training instead of repairing violations after inference [45]. This comparison should be separated from the deterministic clipping used later in dispatch screening: Table 4 tests training-time feasibility guidance against a matched clip-only control, whereas Section 4.3 uses clipping only as part of the shared downstream interface. The hardware rows then place that safety result inside the controller-side screening envelope; only Ultra-LSNT, Ultra-LSNT-Lite, and DLinear are direct same-stack batch-1 measurements, while archived and backfilled rows remain contextual references.

Table 5: Fixed-interface dispatch-screening outcomes under 120-day rolling and RTS-24 DC-OPF evaluation.

Panel A

Fixed-interface comparability / fallback gate

Model	Raw opt.	Raw FB (%)	Mapped opt.	Mapped FB (%)
DLinear	0.0000	100.0	0.3750	62.5
TimeMixer	0.0000	100.0	0.7833	21.7
iTransformer	0.0000	100.0	0.8333	16.7
Ultra-LSNT	0.0000	100.0	0.7583	24.2

Panel B

L1 mapped-protocol outcomes (copper-plate)

Model	L1 Cost (M CNY)	L1 Curt. (M kWh)	L1 FB (%)
iTransformer	32.16	15.22	16.7
TimeMixer	34.90	19.60	21.7
Ultra-LSNT	35.97	22.09	24.2
DLinear (base)	54.47	55.71	62.5

Panel C

L2 mapped-protocol outcomes (RTS-24 DC-OPF)

Model	L2 Curt. (MWh)	L2 Succ. (%)	L2 Gain (%)
iTransformer	138,093.39	100.00	+0.94
TimeMixer	138,093.39	100.00	+0.94
Ultra-LSNT	138,093.39	100.00	+0.94
DLinear (base)	139,403.69	100.00	+0.00

Note: Panel A reports raw-vs-mapped optimization and fallback rates. Panels B and C report fixed-interface, mapped-and-clipped comparative screening outcomes under that same archived interface. Panel C is a coarse network-constrained DC-screen layer; once success saturates, discrimination is limited. Broader mapping-sensitivity evidence is moved to the supplementary material.

5.3. Dispatch screening results

After one fixed mapping and clipping rule is applied to all outputs, iTransformer yields the lowest L1 cost and fallback rate, TimeMixer follows, and Ultra-LSNT still improves on DLinear in cost, curtailment, and fallback frequency. The raw-scale runs are reported once as a closure check: all four models remain in `fallback_x0` for all 120 days before mapping. In the RTS-24 DC-OPF supplement, all mapped models remain DC-feasible, so the network layer mainly acts as an admissibility check with little ranking resolution.

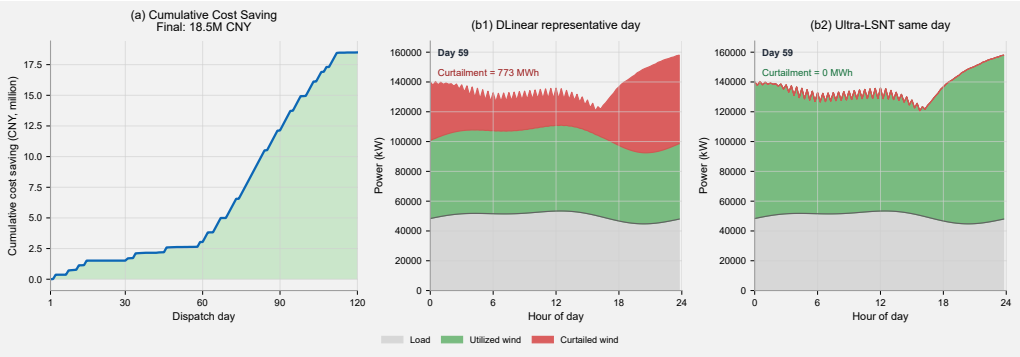


Figure 5: Fixed-interface dispatch screening under the shared mapped-and-clipped interface. Panel (a) shows 120-day cumulative cost saving against DLinear. Panels (b1) and (b2) show the representative day for DLinear and Ultra-LSNT, respectively.

Figure 5 shows the same ordering over time. iTransformer keeps the strongest L1 outcome, TimeMixer follows, and Ultra-LSNT remains materially better than DLinear. In the RTS-24 supplement, 100% DC success across all mapped models compresses ranking resolution, so the network layer mainly confirms that the mapped trajectories remain admissible under the coarse linear screen [2, 28, 29].

5.4. Hyperparameter Pareto and operating points

The P2 sweep serves as an operating-point diagnostic inside Ultra-LSNT. Low λ_2 keeps severe-noise R^2 near 0.93 but leaves non-zero rated-power exceedance, while $\lambda_2 = 1.0$ removes exceedance at large accuracy cost. Figure 6 shows the same trade-off at the anchor setting $(N, K, \tau) = (8, 2, 0.5)$ and highlights the practical safety-accuracy frontier within P2.

6. Discussion and limitations

6.1. Why sparse routing helps under corrupted SCADA

SCADA corruption is heterogeneous over time: missing segments, change points, and failure-related drifts are concentrated rather than uniform [17, 18, 20, 46]. In that setting, a dense always-on pathway allocates the same computation to stable and unstable intervals. Ultra-LSNT instead lets harder windows activate more capacity while easier windows traverse a smaller path. Under matched-control training, all models benefit from fault augmentation, but Ultra-LSNT

Table 6: Pareto operating points in (K, τ, λ_2) at $\sigma_{\text{eff}} = 0.60$ under protocol P2 (SOA proxy).

K	τ	λ_2	**Latency (ms)**	R^2@0.60	**Rated-power exceed. (SOA proxy, %)**
1	0.7	0.01	10.609	0.9301	19.1332
1	0.9	0.10	10.232	0.9257	18.0169
2	0.5	0.01	14.353	0.9294	19.0091
4	0.9	0.01	19.769	0.9299	18.9690
1	0.9	1.00	10.232	0.3831	0.0000
4	0.5	1.00	18.182	0.3913	0.0000

Note: Values are sampled from the archived P2 sensitivity sweep. The reported safety axis is rated-power exceedance under the protocol-P2 SOA proxy and should not be numerically compared with the protocol-P1 values in Table 4.

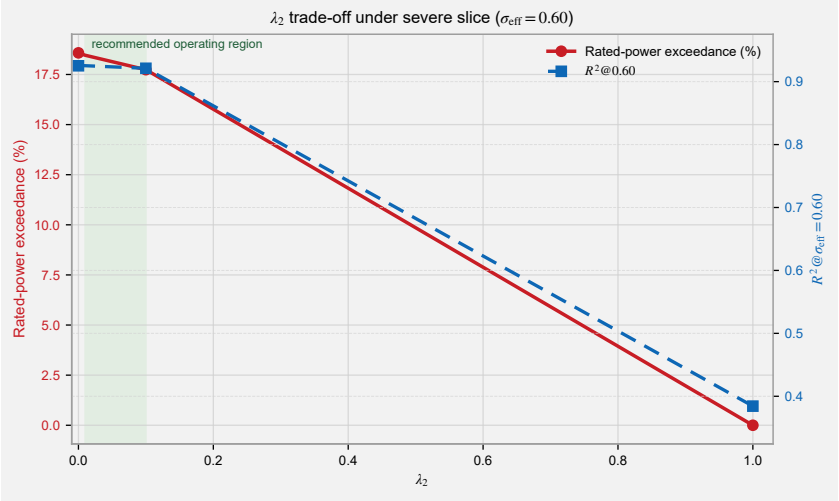


Figure 6: λ_2 safety-accuracy trade-off at anchor $(N, K, \tau) = (8, 2, 0.5)$ under the severe slice $\sigma_{\text{eff}} = 0.60$. The left axis reports rated-power exceedance ($\hat{y} > P_{\text{rated}}$, %) under protocol P2, and the right axis reports R^2@0.60. The shaded band indicates the recommended operating region.

starts from a higher severe-slice baseline and stays positive across the full stress trajectory. That pattern points to adaptive capacity allocation as the likely source of the gain, even though the present evidence does not isolate the router in a causal experiment.

6.2. *Why FG-MoE changes the forecast distribution*

FG-MoE differs from clip-only repair because the envelope enters training, not only the output projection. Recent wind power-curve preprocessing and physics-guided forecasting studies

use physical priors to reshape learning before inference [21, 26, 33]. Table 4 makes the comparison concrete: clip-only removes exceedance but cuts clean and severe-slice R^2 to 0.4319 and 0.3120, whereas FG-MoE keeps 0.8747 and 0.8286 with exceedance at 0.0092%. The relevant distinction is therefore not whether outputs can be clipped after prediction, but whether training shifts the forecast distribution toward feasible operating regions without sacrificing the main accuracy regime. The last-observation wind proxy is central to that trade-off: it keeps the guardrail deployable at inference, but it can underrepresent forward-looking wind evolution and therefore sacrifice some anticipatory discipline during rapid regime changes.

6.3. Dispatch evidence under a fixed interface

Dispatch value remains tied to the chosen interface. Coordinated scheduling and economic-value studies already show that forecast quality maps to different reserve, curtailment, and cost outcomes under different market or system models [1, 3, 4]. Under the fixed mapping used here, iTransformer has the lowest L1 cost and fallback rate, while Ultra-LSNT still lowers L1 cost and fallback days relative to DLinear after raw forecasts fail to close the 120-day runs directly. The important point is that one shared mapping still separates stronger from weaker local models, so the dispatch evidence should be read as comparative screening under that interface rather than as model-intrinsic value.

RTS-24 adds a coarse network check. Once all mapped models clear the linear DC screen, Panel C no longer resolves fine ranking differences; it only shows that the compared trajectories remain admissible under that approximation. AC feasibility and fuller market validation remain outside the scope of this screen.

6.4. External validity and remaining limits

Several limits remain explicit. The main forecasting evidence still comes from one operational Wind (CN) site, so site-to-site heterogeneity is not exhausted. The corruption protocol is operationally motivated but not purely observational: it combines recorded SCADA irregularities with synthetic injection and stress-envelope design, so it does not span the full space of real fault processes. The feasibility regularizer is also intentionally deployable rather than fully foresighted: the last-observation wind proxy can be evaluated online, but that choice reduces forward visibility under fast regime shifts.

Several additional limits matter for interpretation. External validation remains limited: DC feasibility on RTS-24 does not imply AC feasibility [30], and the same mapped interface can compress downstream differences once all models pass the same DC success gate. The hardware profile mixes direct batch-1 measurements with archived or backfilled reference rows and remains pre-HIL rather than board-level evidence on target PLC/RTU hardware; controller-side SCADA infrastructure studies remain broader than the measurement envelope reported here [47]. P1 safety values in Table 4 and P2 SOA-proxy values in Table 6 also come from different protocols and should not be compared directly.

7. Conclusion and outlook

This study frames short-term wind forecasting as a stricter controller-side problem in which corrupted SCADA, batch-1 edge budgets, deployable feasibility signals, and a fixed forecast-to-dispatch interface must be considered jointly. Under that protocol, the evidence supports three bounded conclusions. First, a sparse controller-side design can remain usable under severe corruption rather than collapsing with the input stream. Second, feasibility guidance is most effective when it reshapes training toward admissible operating regions instead of relying only on posthoc clipping. Third, forecast value should be interpreted through a shared downstream screen, because the reported differences are dispatch-screening differences under a shared interface rather than model-intrinsic gains. These findings therefore support a bounded comparative result under the present corrupted-SCADA, fixed-interface, pre-HIL protocol rather than operator-grade proof: the main forecasting evidence comes from one Wind (CN) site, the corruption protocol remains partly synthetic, the dispatch layer is a comparative screening workflow, and the hardware profile is still pre-HIL. The next step is therefore external validation rather than broader claiming, including additional sites, less synthetic fault injection, AC-grade operator studies, and board-level measurements on target controller hardware.

CRediT authorship contribution statement

Junyu Li: Conceptualization, Methodology, Software, Validation, Formal analysis, Data curation, Writing – original draft, Visualization. **Juntao Du**: Supervision, Writing – review & editing, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Wind (CN): The Wind (CN) SCADA dataset used in this study is released publicly to support full reproducibility. The release includes the raw time series, the train/validation/test split indices, and the preprocessing scripts used to construct the learning task described in Table 2. The dataset package is distributed via the project repository (https://github.com/b1ue13e/Ultra-LSNT) and will be mirrored to a long-term archival service at acceptance.

Public benchmarks: The additional datasets used for cross-domain evaluation are publicly accessible: (i) Wind (US) from the NREL Wind Toolkit [48], (ii) Air Quality from the UCI repository [49], and (iii) GEFCom load from the Global Energy Forecasting Competition (GEFCom2012) [50]. We report the exact dataset identifiers, preprocessing scripts, and evaluation protocols in the released code.

In line with Elsevier's research data practices, we provide reproducibility assets (protocol definitions, supplementary evidence tables, and run manifests) through the project repository and archived supplementary evidence bundle.

Code: The implementation, training scripts, and evaluation protocols are available at https://github.com/b1ue13e/Ultra-LSNT (MIT license).

Appendix A. Nomenclature and abbreviations

Item	Description
Abbreviations	
MoE	Mixture-of-Experts
SCADA	Supervisory Control and Data Acquisition
TSFM	Time-series foundation model
SOTA	State-of-the-art
RMSE	Root Mean Square Error
MAE	Mean Absolute Error
FG-MoE	Feasibility-Guided MoE (main-text term in this manuscript)

Item	Description
UQ	Uncertainty Quantification
CRPS	Continuous Ranked Probability Score
AC-OPF	AC Optimal Power Flow
DC-OPF	DC Optimal Power Flow
HIL	Hardware-in-the-Loop
Symbols	
L	input length (time steps)
H	prediction horizon (time steps)
C	number of input variables/covariates
$\mathbf{X} \in \mathbb{R}^{L \times C}$	input sequence
$\hat{\mathbf{Y}} \in \mathbb{R}^{H \times 1}$	forecast output (wind power / target variable)
σ_{cfg}	configured perturbation ratio in robustness tests (dimensionless)
σ_{eff}	effective perturbation ratio, $\sigma_{\text{eff}} = \alpha\sigma_{\text{cfg}}$
α	scaling factor used to map configured to effective noise severity ($\sigma_{\text{eff}} = \alpha\sigma_{\text{cfg}}$, default $\alpha = 1.5$)
K	number of selected experts in Top-K routing
λ_r	load-balancing regularization coefficient for routing
τ	Jump Gate threshold for conditional skipping
R^2	coefficient of determination (scale-free)
RMSE/MAE	absolute errors in original physical units after inverse scaling
E_{inf}	estimated energy per sample inference (mJ)
B	batch size used in profiling (dimensionless)
N_{call}	number of forecasting calls per day
N_{turb}	number of turbines in a wind farm or fleet
c_f	fuel / purchase price coefficient in dispatch (CNY/kWh)
c_r	reserve price coefficient (CNY/kWh)
c_s	storage discharge cost coefficient (CNY/kWh)
c_{su}	start-up / shut-down cost coefficient (CNY)
$P_{\text{ch,max}}$	storage charging power limit (kW)
$P_{\text{dis,max}}$	storage discharging power limit (kW)
S_{max}	storage energy capacity limit (kWh)
R_{max}	thermal ramp-rate limit (kW per hour)
$\bar{P}_{\text{dev}}$	average device power during inference (W)
ρ	air density (kg/m^3), estimated from temperature/pressure
A	rotor swept area (m^2)
v	wind speed at hub height (m/s)
v_{t+h}^{proxy}	deployable wind proxy used by FG-MoE, with $v_{t+h}^{\text{proxy}} = v_t$
$C_p(\lambda_{\text{tsr}}, \beta)$	power coefficient (Betz-limited)
λ_{tsr}	tip-speed ratio (dimensionless)
β	pitch angle (deg)
P_{rated}	rated-power upper bound (kW)

Item	Description
$P_{\mathrm{env}}(v_{t+h}^{\mathrm{proxy}})$	turbine envelope evaluated at the deployable wind proxy (kW)
$v_{\mathrm{in}}, v_{\mathrm{out}}$	cut-in / cut-out wind speeds (m/s)
λ_2	plausibility-penalty coefficient in heuristic FG-MoE (dimensionless)

Appendix B. Dispatch model details

This appendix provides the deterministic dispatch algebra used by the screening workflow in Section 4.3. The formulation is included for reproducibility and serves as comparative screening infrastructure rather than AC-secure operational guidance.

With horizon $T = 24$ and $\Delta t = 1$ h, the objective is

$$\min \; C_{\mathrm{dispatch}} = C_{\mathrm{fuel}} + C_{\mathrm{reserve}} + C_{\mathrm{purchase}} + C_{\mathrm{storage}} + C_{\mathrm{startup}} + C_{\mathrm{shutdown}}, \tag{B.1}$$

$$C_{\mathrm{fuel}} = c_f \sum_{t=1}^{T} g_t \Delta t,$$

$$C_{\mathrm{reserve}} = c_r \sum_{t=1}^{T} r_t \Delta t,$$

$$C_{\mathrm{purchase}} = c_f \sum_{t=1}^{T} p_{\mathrm{ch},t} \Delta t,$$

$$C_{\mathrm{storage}} = c_s \sum_{t=1}^{T} p_{\mathrm{dis},t} \Delta t,$$

$$C_{\mathrm{startup}} = c_{\mathrm{su}} \sum_{t=2}^{T} \frac{\max(g_t - g_{t-1}, 0)}{R_{\max}},$$

$$C_{\mathrm{shutdown}} = c_{\mathrm{su}} \sum_{t=2}^{T} \frac{\max(g_{t-1} - g_t, 0)}{R_{\max}}.$$

Subject to

$$g_t + \hat{w}_t + p_{\mathrm{dis},t} - p_{\mathrm{ch},t} - u_t = d_t, \tag{B.2}$$

$$0 \le u_t \le \hat{w}_t, \quad \forall t,$$

$$g_{\min} \le g_t \le g_{\max}, \tag{B.3}$$

$$0 \le r_t, \quad g_t + r_t \le g_{\max},$$

$$|g_t - g_{t-1}| \le R_{\max},$$

$$0 \le p_{\mathrm{ch},t} \le P_{\mathrm{ch,max}}, \tag{B.4}$$

$$0 \le p_{\mathrm{dis},t} \le P_{\mathrm{dis,max}},$$

$$s_t = s_{t-1} + \eta_{\mathrm{ch}} p_{\mathrm{ch},t} \Delta t - \eta_{\mathrm{dis}}^{-1} p_{\mathrm{dis},t} \Delta t, \tag{B.5}$$

$$0 \le s_t \le S_{\max}.$$

and the fixed-proportional reserve protocol

$$\sigma_{w,t} = 0.2 \cdot |\hat{w}_t|, \ t = 1, \ldots, T. \tag{B.6}$$

which is a deterministic comparison rule and not UQ-aware dynamic reserve optimization.

Appendix C. Implementation and environment details

Appendix C.1. Implementation details and computational environment

All experiments are implemented in Python (v3.10) using PyTorch (v2.0+) with CUDA (v11.8) acceleration. The core scientific stack includes NumPy (v1.24+) and Pandas (v2.0+) for data handling, Matplotlib (v3.7+) and Seaborn (v0.12+) for visualization, and LightGBM (v4.0+) with Scikit-learn (v1.3+) for tree-based baselines. These versions are reported to ensure that both deep and non-deep baselines can be reproduced under consistent dependencies. The implementation is openly available at https://github.com/b1ue13e/Ultra-LSNT (MIT license), enabling inspection of model components (e.g., sparse routing, skipping gate) and exact reproduction of the reported protocols.

The main training is conducted on a workstation equipped with an NVIDIA GeForce RTX 4090 GPU, an Intel i9-13900K CPU, and 64 GB RAM for reproducible optimization and ablation sweeps. To complement workstation profiling, we report batch-1 inference latency on

commodity CPUs and an ARMv8 (aarch64) single-core platform (Huawei Kunpeng 920). The Kunpeng run is used as a server-class ARM reference for cross-ISA comparison rather than as an edge-device surrogate.

Implementation and training: We implement Ultra-LSNT in native PyTorch without third-party model dependencies. The sparse MoE core uses a custom Top-K routing layer ($K = 2$ in the main model), and the dynamic skip path uses a Jump Gate style conditional pass. To stabilize routing, the released configuration sets the auxiliary load-balancing coefficient to $\lambda = 0.01$ and keeps the router z-loss at 0.01, following standard MoE load-balancing and router-stabilization practice [38]; a controlled $\lambda \in \{0.01, 0.05\}$ robustness check is summarized in Table 4. Unless otherwise stated, we train with $L = 96$, $H = 24$, chronological splits, train-fitted z-score normalization, AdamW $(\beta_1, \beta_2) = (0.9, 0.999)$, weight decay 0.01, cosine annealing with warm-up, initial learning rate 10^{-3}, and batch size 128; typical runs last 10–50 epochs depending on convergence.

Deployment profiling and reproducibility: We profile inference under fixed batch-1 settings for x86/ARM latency screening (Table 4) and fix seeds plus deterministic settings when possible. Ultra-LSNT, Ultra-LSNT-Lite, and DLinear are directly profiled under the released stack, whereas several additional rows are retained as archived or backfilled screening references rather than as same-stack head-to-head measurements. Low-level runtime switches, archived manifests, and environment-specific optimization details are documented in the supplementary material rather than repeated in the main manuscript.

Appendix D. Robustness protocol details

Appendix D.1. Perturbation definitions and comparison setup

Additive Gaussian noise: We use additive Gaussian corruption as a controlled sensor-inaccuracy stress test, capturing short-lived measurement noise such as electromagnetic interference and turbulent sensing jitter in SCADA channels. Formally, for a clean input series x, we generate a corrupted series

$$x_{\text{noisy}} = x + \sigma_{\text{eff}}\,\text{std}(x) \cdot \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, 1), \tag{D.1}$$

where $\text{std}(x)$ is the standard deviation of the clean signal and σ_{eff} is the effective noise ratio. In our released noise generator, we use an industrial scaling factor $\alpha = 1.5$. To strengthen

empirical grounding, we additionally compute a channel-wise proxy noise ratio on Wind (CN) core channels (windspeed, winddirection, temperature, humidity, pressure, power):

$$r_c = \frac{\mathrm{std}(\Delta x_c)}{\sqrt{2}\,\mathrm{std}(x_c)}, \tag{D.2}$$

where $\Delta x_c = x_{t,c} - x_{t-1,c}$. On this dataset, the proxy statistics are median 0.045, P90 0.091, and max 0.121. Detailed calibration artifacts are archived in the supplementary evidence bundle.

Based on this anchor, we treat $\sigma_{\mathrm{eff}} = 0.15$ as an empirical upper-band neighborhood, while $\sigma_{\mathrm{eff}} \in \{0.30, 0.45, 0.60\}$ are explicit stress-envelope settings for robustness certification under abnormal SCADA conditions rather than nominal sensor-error claims. Throughout the manuscript, severity is reported by the *effective* ratio $\sigma_{\mathrm{eff}} = \alpha\sigma_{\mathrm{cfg}}$, so the injected noise has standard deviation $\sigma_{\mathrm{eff}} \cdot \mathrm{std}(x)$. In configuration files and scripts, the user-facing parameter is σ_{cfg}; we therefore report both σ_{cfg} and σ_{eff} when needed. Unless otherwise stated, the Gaussian sweep uses $\sigma_{\mathrm{eff}} \in \{0.00, 0.15, 0.30, 0.45, 0.60\}$ (equivalently $\sigma_{\mathrm{cfg}} \in \{0.00, 0.10, 0.20, 0.30, 0.40\}$).

Structured SCADA faults: Beyond Gaussian noise, real SCADA failures usually show temporal persistence and cross-channel dependence. To reflect that behavior, we synthesize three structured fault families on top of clean streams: flatlining (sensor-freeze windows generated by dwell-time processes such as geometric or semi-Markov sampling), spikes/heavy-tailed outliers (sparse impulsive corruption with amplitudes from Laplace or Student-t distributions), and correlated multi-channel dropouts (common-cause masking events such as DAQ resets that affect multiple variables jointly for contiguous windows). We evaluate these faults on Wind (CN) at severities 0.2 and 0.6; the severe slice is summarized in Table 3, while full multi-severity rows are moved to supplementary material.

To complement deep-model comparisons, we also run focused non-deep context baselines (LightGBM and Persistence) over configured noise $\sigma_{\mathrm{cfg}} \in \{0.00, 0.10, 0.20, 0.30, 0.40\}$ (effective $\sigma_{\mathrm{eff}} \in \{0.00, 0.15, 0.30, 0.45, 0.60\}$ with $\alpha = 1.5$), so severe-slice behavior is inspected with both feature-engineered and naive-history references.

Drift and quantization noise: We also examine two perturbation families that arise naturally in industrial sensing and edge pipelines. Drift noise approximates systematic bias that accumulates over time due to calibration drift, sensor aging, or persistent environmental interference, whereas quantization noise captures rounding and discretization errors introduced by low-precision telemetry, bandwidth-constrained transmission, or edge-side INT8/FP16 processing.

To keep perturbation severity comparable, we report both perturbations using the same effective ratio σ_{eff} as in the Gaussian protocol. Unless otherwise stated, we sweep

$$\sigma_{\text{eff}} \in \{0.00, 0.15, 0.30, 0.45, 0.60\}, \tag{D.3}$$

which corresponds to configured $\sigma_{\text{cfg}} \in \{0.00, 0.10, 0.20, 0.30, 0.40\}$ when $\alpha = 1.5$.

Drift (random-walk bias): Given a clean channel $x_{1:L}$, we draw i.i.d. $\varepsilon_t \sim \mathcal{N}(0, 1)$ and define a persistent bias process

$$b_1 = 0. \tag{D.4}$$

$$b_t = b_{t-1} + \sigma_{\text{eff}} \cdot \text{std}(x) \cdot \varepsilon_t, \; t = 2, \ldots, L. \tag{D.5}$$

then corrupt the input as $x_t^{\text{drift}} = x_t + b_t$. The process is applied independently to each feature channel and each test sample. We reset $b_1 = 0$ per sample for paired reproducibility, and use the same random seed across methods at a given σ_{eff}; long contiguous fault persistence is covered separately by the structured-fault protocol.

Quantization (step-size rounding): We implement quantization as deterministic rounding to a channel-wise step size proportional to the clean-signal standard deviation:

$$\Delta = \sigma_{\text{eff}} \cdot \text{std}(x). \tag{D.6}$$

$$x_t^{\text{quant}} = \Delta \cdot \text{round}\left(\frac{x_t}{\Delta}\right). \tag{D.7}$$

where round$(\cdot)$ is round-to-nearest (half away from zero). When $\sigma_{\text{eff}} = 0$, no quantization is applied. This protocol yields a reproducible, severity-controlled discretization without requiring dataset-specific bit-range tuning. We report the corresponding robustness outcomes in Section 5.1 and interpret their operational implications in the Discussion, with emphasis on reliability under realistic sensing constraints and energy-aware (Green AI) deployment.

Missingness and interpolation artifacts: Communication loss and sensor resets regularly create gaps in SCADA telemetry, after which simple forward-fill or linear interpolation is often applied. To emulate this setting, we sample a binary mask $m_{t,c} \sim \text{Bernoulli}(1 - p)$ over time steps t and channels c and define $\bar{x}_{t,c} = m_{t,c} x_{t,c} + (1 - m_{t,c})\text{NaN}$. We then apply an imputation operator Imp$(\cdot)$ to obtain the corrupted series

$$x^{\text{miss}} = \text{Imp}(\bar{x}; \texttt{ffill/linear}), \tag{D.8}$$

where `ffill` denotes forward filling and `linear` denotes piecewise-linear interpolation along the time axis. In the reported experiments, we use i.i.d. masking (no burst blocks) as a controlled imputation stress test, sweep $p \in \{0.1, 0.3\}$, and repeat each configuration for three random seeds (0,1,2). For a fixed seed, the same missingness mask and imputation configuration are applied across models to preserve a paired comparison. Contiguous and correlated missingness is represented by the structured-fault dropout protocol. Results are provided in supplementary material.

Appendix E. Additional deployment and sensitivity figures

Figures E.7–E.10 address four supplementary questions: where deployment bottlenecks emerge under severe corruption, how efficiency axes co-vary, how Jump Gate threshold shifts the accuracy-latency trade-off, and how the structural N-K-τ landscape behaves at fixed λ_2. Figure E.7 isolates corruption-level deployment bottlenecks, Figure E.8 visualizes accuracy-efficiency coupling, Figure E.9 tracks threshold-driven accuracy-latency shifts, and Figure E.10 summarizes structural Pareto geometry. For Figure E.7, severe-slice R^2 follows the matched-control track and is paired with Panel-A CPU latency from Table 4 as supplementary deployment context.

Appendix F. Protocol scope and practical limits

Dispatch scope: Main-text dispatch results use one archived mapped-and-clipped forecast-to-grid interface. The RTS-24 layer is a coarse DC screening supplement rather than an operator-grade ranking environment. Supplementary material retains mapping-sensitivity checks, and no interface-independent dispatch claim is made.

Hardware scope: Cross-ISA latency profiling is mixed-provenance and pre-HIL. Some rows are directly measured, others are archived/backfilled references, and board-level validation on target PLC/RTU hardware remains required.

Matched-control scope: This revision adds matched clean/fault-augmented controls for DLinear, TimeMixer, iTransformer, and Ultra-LSNT (three seeds; four faults), but full coverage for long-tail supplementary baselines remains future work.

Transfer scope: Cross-domain transfer results are retained as secondary diagnostic evidence in Table G.9. They are used only to show that no single model dominates all tasks and interfaces.

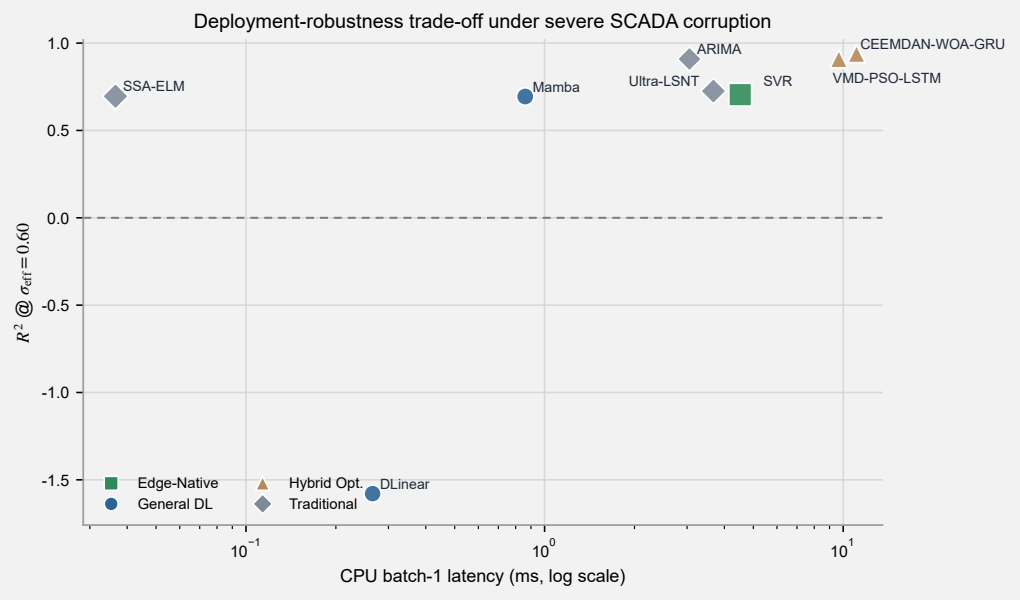


Figure E.7: Deployment-robustness trade-off under severe SCADA corruption ($\sigma_{\text{eff}} = 0.60$). CPU batch-1 latency is shown on the x-axis (log scale), and R^2@0.60 is shown on the y-axis. Point size encodes model size, and colors/markers indicate model family.

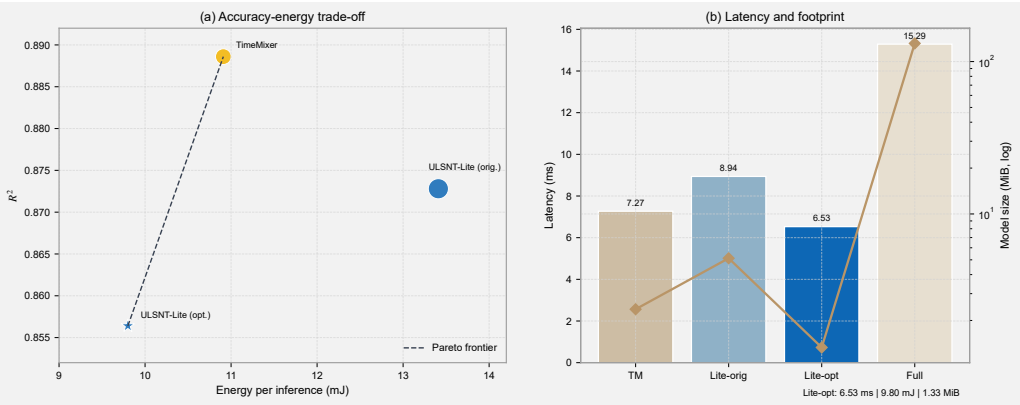


Figure E.8: Deployment-efficiency comparison across Ultra-LSNT variants and principal baselines. Energy, latency, and memory are reported jointly as edge-suitability indicators.

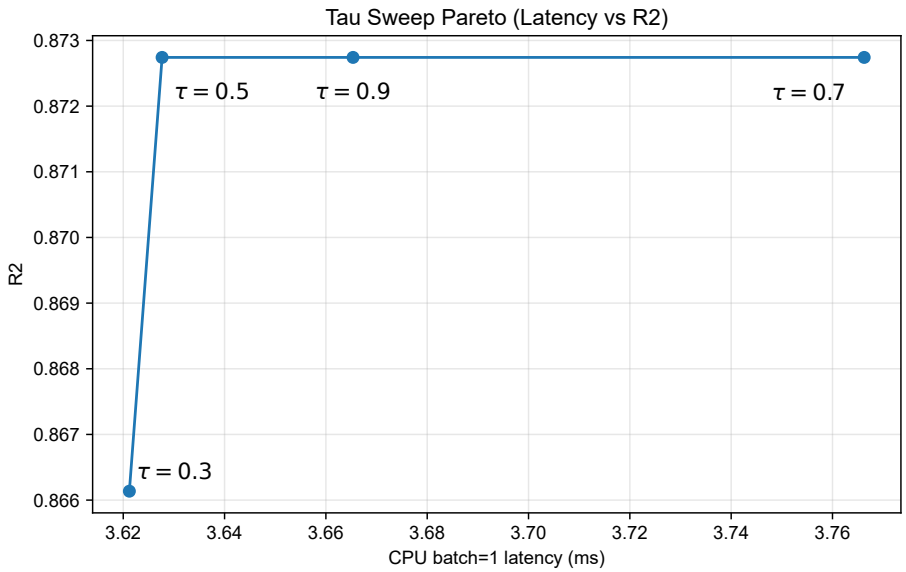


Figure E.9: Jump Gate threshold sweep in supplementary view. The threshold-latency-accuracy trajectory should be read jointly with the archived skip-ratio logs.

Appendix G. Supplementary tables: ablations, interpretability, and significance

Appendix G.1. Evidence-file mapping for primary comparative numbers

Primary comparative numbers in this manuscript are generated from consolidated audit tables in the archived supplementary evidence bundle rather than by ad hoc extraction. Detailed file manifests are retained in the public repository and supplement; the main text only relies on those consolidated tables for robustness, safety, and shared-interface dispatch screening, while transfer results are retained as secondary diagnostic evidence.

Appendix G.2. Scale-error supplementary table for robustness audit

Table G.8 provides supplementary RMSE/MAE evidence for the Wind (CN) context and matched-control rows used in this revision. The main table already reports RMSE in Panel A; Appendix G retains MAE and protocol-specific row coverage for completeness rather than as a substitute for the primary comparison.

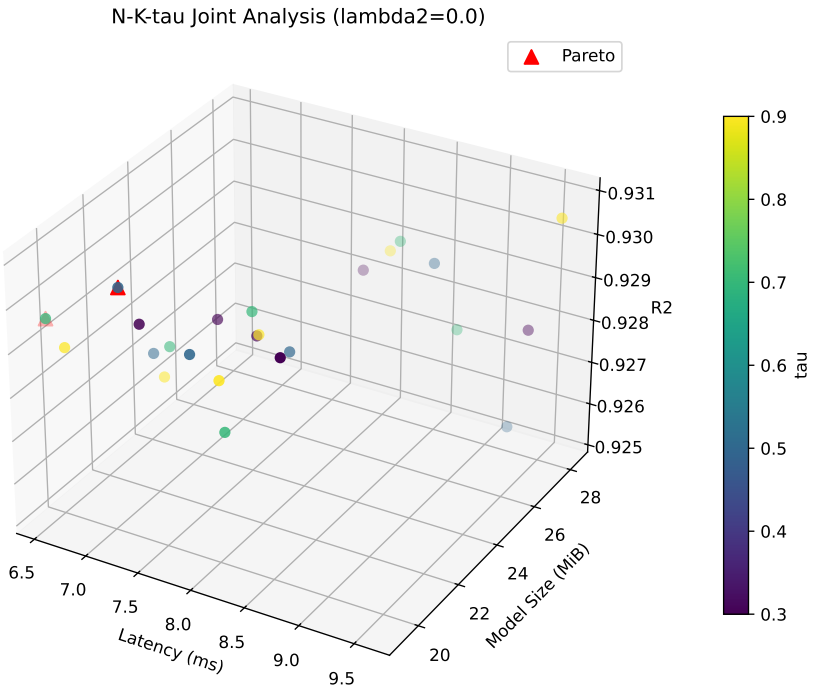


Figure E.10: 3D structural map (N-K-τ) at $\lambda_2 = 0$ with Pareto points highlighted (Wind CN, $\sigma_{\text{eff}} = 0.60$). Main-text decision evidence is reported in Table 6 and Figure 6.

Appendix G.3. Secondary diagnostic transfer table

Table G.9 retains the cross-domain transfer audit under the same chronological protocol used in the archived evidence bundle. It is reported as secondary diagnostic evidence rather than as part of the main Wind (CN) claim chain.

Appendix G.4. Full structured SCADA fault robustness table

Complete multi-severity and multi-metric structured-fault results for Wind (CN) are reported in supplementary material and archived in the supplementary evidence bundle.

Appendix G.5. Diagnostic long-tail baseline matrix (diagnostic-only)

For transparency, we retain broad baseline rows, including traditional methods, hybrid methods, and constrained-budget metaheuristics, in the supplementary evidence bundle. These rows are diagnostic only and are excluded from the matched-control main comparisons.

Table G.8: Supplementary scale-error evidence at clean and $\sigma_{\text{eff}} = 0.60$ for directly logged Wind (CN) rows.

Model	Mode	RMSE (clean)	MAE (clean)	RMSE@0.60	MAE@0.60
DLinear	clean-trained	6065.98	3865.18	22190.19	15183.13
TimeMixer	clean-trained	9081.04	6789.63	12848.91	9674.21
iTransformer	clean-trained	7743.06	6029.31	9933.03	7852.08
Ultra-LSNT	clean-trained	9462.65	7360.92	10992.42	8705.76
DLinear	fault-augmented	6328.26	4181.08	20569.46	14105.63
TimeMixer	fault-augmented	8969.46	7067.89	10129.43	8034.08
iTransformer	fault-augmented	8779.48	6975.59	9593.71	7700.04
Ultra-LSNT	fault-augmented	9925.64	7858.85	10859.03	8675.17
Persistence	clean-trained	7313.94	4760.16	9485.55	6783.51
LightGBM	clean-trained	6880.91	4622.43	17665.50	13740.79
Chronos-Bolt-Small	zero-shot (512 ctx)	6216.19	3818.52	11443.18	8407.16
Moirai-2.0-R-Small	zero-shot (512 ctx)	5834.04	3460.46	11045.80	8176.62

Note: Matched-control rows for DLinear, TimeMixer, iTransformer, and Ultra-LSNT should be read as Panel B companions rather than as Table 3 Panel A context reruns. Chronos and Moirai use zero-shot extended-context inference (512-step input, same 24-step target window) and do not support controller-side deployability claims. Detailed source manifests are provided in the supplementary evidence bundle.

Table G.9: Secondary diagnostic evidence: cross-domain transferability under a unified chronological protocol.

Model	Wind (US) R^2	Air Quality R^2	GEFCom Load R^2
Ultra-LSNT	0.7878	**0.4861**	0.7554
DLinear	0.8779	0.3433	0.7370
iTransformer	0.8563	0.4590	0.7467
TimeMixer	0.8023	0.3869	0.7201
ARIMA (logged)	0.6971	0.2008	−0.4031
SVR (logged)	0.5796	0.1716	0.8226
SSA-ELM	0.8075	0.3152	0.7662
Mamba (logged)	0.8866	0.3688	−0.0083

Note: Values are taken from the archived supplementary evidence bundle under the same chronological protocol and should be read as boundary-setting diagnostic evidence rather than as an extension of the main matched-control claim.

Appendix G.6. Ablation results

All ablations are conducted under a fixed training budget and identical evaluation protocol; we interpret them primarily through relative ranking and directional sensitivity, rather than expecting the absolute R^2 to exactly match the full benchmark runs.

Supplementary ablation tables report kernel-size sensitivity from the archived ablation CSV.

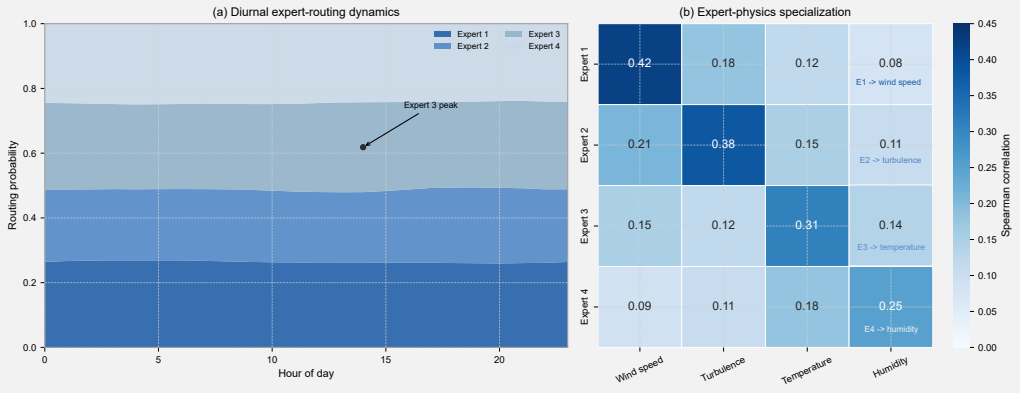


Figure G.11: Interpretability view of sparse routing and expert-feature correlation. The left panel reports diurnal expert-routing composition, and the right panel reports the expert-feature correlation heatmap.

On Wind (CN), $k = 3$ and $k = 7$ are close, while $k = 15$ is weaker. This supports using compact receptive fields for turbulence-dominant SCADA dynamics. Router-regularization robustness is reported in Table 4 using the two archived λ checkpoints (0.01 and 0.05).

Appendix G.7. XAI analysis: routing diagnostics for operational audit

Figures G.11 and G.12 visualize sparse-routing statistics at complementary scales: diurnal/feature correlation and event-window temporal dynamics. This subsection is exploratory governance evidence, not proof of causal physical decomposition. Because routing is regularized by load-balancing objectives, expert activations should be interpreted as correlation-level diagnostics instead of deterministic physical role assignment.

At the diurnal scale, activation shares remain bounded without collapse spikes, which supports auditability of conditional computation. At event windows, routing redistribution is observable and can be logged as a monitoring signal for anomaly triage, but these shifts alone should not be interpreted as direct physical regime identification.

Appendix G.8. Statistical significance

We assess whether performance gaps are statistically significant using the Diebold–Mariano (DM) test on squared prediction errors [51]. Let $d_t = \ell_t(\text{Ultra-LSNT}) - \ell_t(\text{Baseline})$ denote the per-time step loss differential computed on the same test split; hence DM< 0 indicates Ultra-LSNT has smaller average loss, while DM> 0 favors the baseline. This significance check is

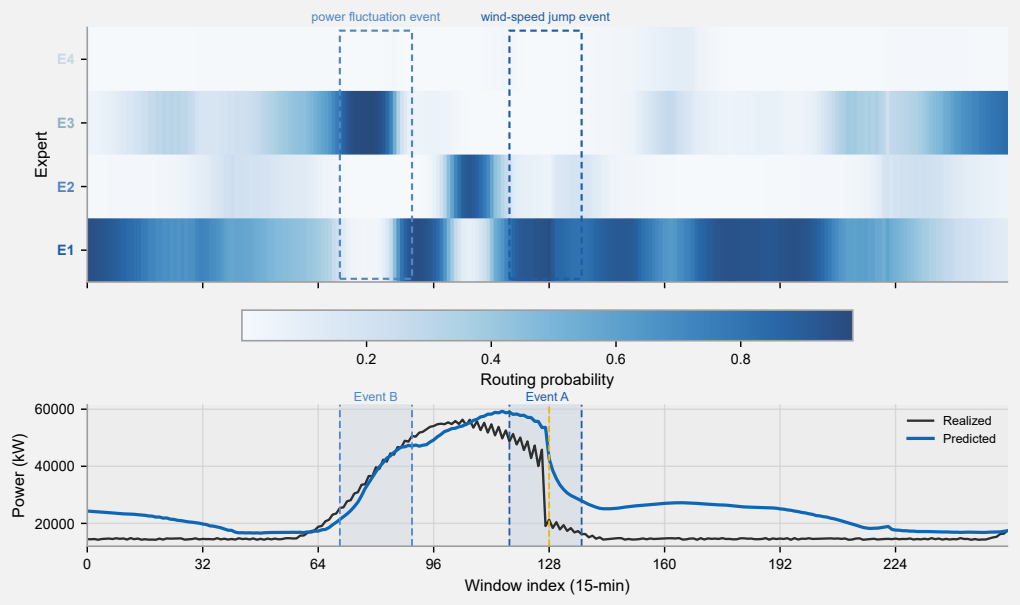


Figure G.12: Time-resolved routing probabilities (4 experts) over a representative 64-hour window with overlaid realized and predicted power. Marked event windows show temporal co-variation between routing redistribution and abrupt trajectory changes; the figure is provided for auditability rather than causal attribution.

reported to strengthen the auditability of our empirical claims, complementing the descriptive ranking in Table 3. Figure G.13 summarizes the four-domain DM significance matrices used in this revision. Readers should treat the DM view as direction-stability evidence (paired loss differential under fixed splits), not as a replacement for protocol-bound robustness and dispatch screening claims in the main results.

Supplementary DM-test tables indicate statistically significant loss differentials across representative recent backbones. Consistent with Table 3, the differentials favor TimeMixer and iTransformer under clean measurements (DM$>$ 0), whereas Ultra-LSNT significantly outperforms PatchTST (DM$<$ 0). We also provide scripts and the loss-differential series for the full set of backbones in the supplementary material.

Several recent forecasters can achieve higher peak accuracy under clean measurements (Table 3), but peak clean-data accuracy is not always aligned with stability under realistic sensing imperfections. Accordingly, we report robustness stress tests and downstream dispatch evidence next, where reliability under noise and deployability constraints matter for renewable integration

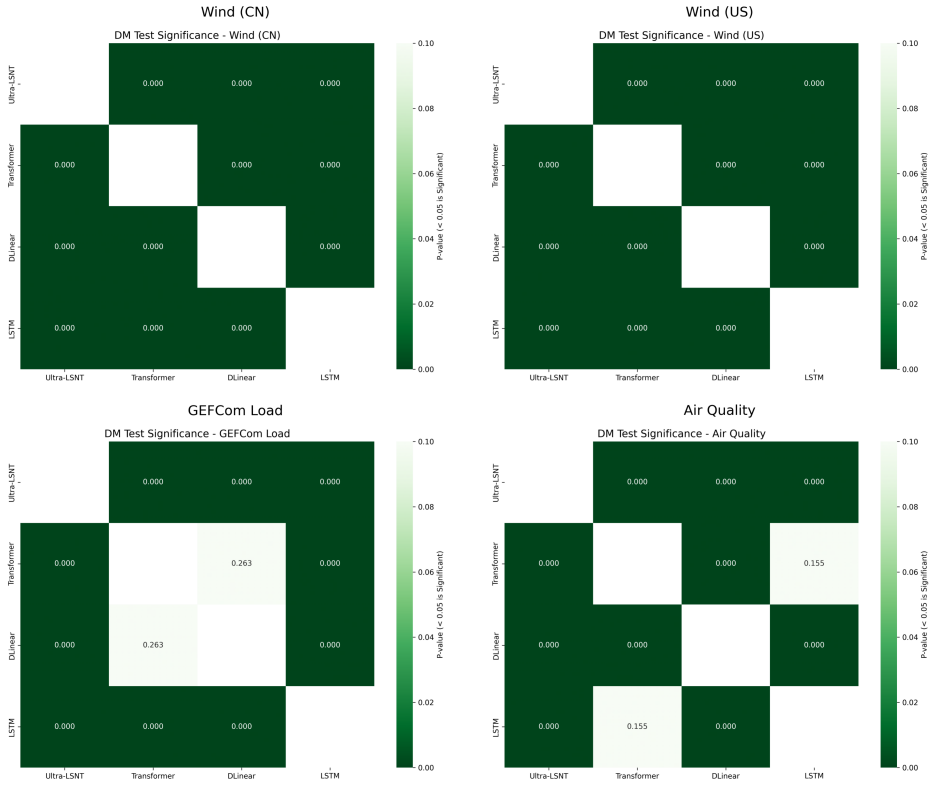


Figure G.13: Direction-aware 2×2 summary of Diebold–Mariano evidence (Ultra-LSNT vs principal baselines) across Wind (CN), Wind (US), GEFCom Load, and Air Quality. Cell entries report direction, p-value, and significance level; complete pairwise DM-statistic magnitude matrices would require dedicated reruns and are not reconstructed here.

and operational efficiency.

Generative AI and AI-assisted technologies statement

During the preparation of this work, the authors used *ChatGPT* (OpenAI) for language polishing and grammar assistance. The authors performed all scientific interpretation, result reporting, and final manuscript decisions, and they take full responsibility for the content of the publication. No generative AI tools were used to generate or modify experimental data, figures, plots, or graphical abstracts.

Acknowledgments

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

References

[1] H. Khaloie, A. Abdollahi, M. Shafie-khah, A. Anvari-Moghaddam, S. Nojavan, P. Siano, J. P. Catalão, Coordinated wind-thermal-energy storage offering strategy in energy and spinning reserve markets using a multi-stage model, Applied Energy 259 (2020) 114168.

[2] N. Houben, A. Cosic, M. Stadler, M. Mansoor, M. Zellinger, H. Auer, A. Ajanovic, R. Haas, Optimal dispatch of a multi-energy system microgrid under uncertainty: A renewable energy community in austria, Applied Energy 337 (2023) 120913. doi:10.1016/j.apenergy.2023.120913.

[3] C. Li, A. Kies, K. Zhou, M. Schlott, O. E. Sayed, M. Bilousova, H. Stöcker, Optimal power flow in a highly renewable power system based on attention neural networks, Applied Energy 359 (2024) 122779. doi:10.1016/j.apenergy.2024.122779.

[4] J. Yan, X. Li, H. Wang, F. Si, W. Qiao, Y. Liu, The economic value of wind power forecasting: a data-driven method and its application in various scenarios, Energy 340 (2025) 139151. doi:10.1016/j.energy.2025.139151.

[5] A. Masood, U. Ahmed, S. Z. Hassan, A. R. Khan, A. Mahmood, Economic value creation of artificial intelligence in supporting variable renewable energy resource integration to power systems: A systematic review, Sustainability 17 (6) (2025) 2599. doi:10.3390/su17062599.

[6] A. Verdone, M. Panella, E. De Santis, A. Rizzi, A review of solar and wind energy forecasting: From single-site to multi-site paradigm, Applied Energy 392 (2025) 126016. doi:10.1016/j.apenergy.2025.126016.

[7] Y. Zhang, K. Liu, J. Qian, Wind power prediction based on variational mode decomposition and artificial intelligence algorithms: A review, Energy Conversion and Management 227 (2021) 113615. doi:10.1016/j.enconman.2020.113615.

[8] W. Duan, Z. Zhang, A. Zhong, Z. Tang, A robust wind power forecasting framework for non-stationary signals via decomposition and metaheuristic optimization, Energies 18 (24) (2025) 6515. doi:10.3390/en18246515.

[9] Y. Zhu, Y. Liu, N. Wang, Z. Zhang, Y. Li, Real-time error compensation transfer learning with echo state networks for enhanced wind power prediction, Applied Energy 379 (2025) 124893. doi:10.1016/j.apenergy.2024.124893.

[10] Z. Ma, G. Mei, A hybrid attention-based deep learning approach for wind power prediction, Applied Energy 323 (2022) 119608. doi:10.1016/j.apenergy.2022.119608.

[11] M. Gao, S. Zhou, W. Gu, Z. Wu, H. Liu, A. Zhou, X. Wang, Mmgpt4lf: Leveraging an optimized pre-trained gpt-2 model with multi-modal cross-attention for load forecasting, Applied Energy 392 (2025) 125965. doi:10.1016/j.apenergy.2025.125965.

[12] T. Wu, Q. Ling, Stellm: Spatio-temporal enhanced pre-trained large language model for wind speed forecasting, Applied Energy 375 (2024) 124034. doi:10.1016/j.apenergy.2024.124034.

[13] J. Maldonado-Correa, S. Martín-Martínez, E. Artigao, E. Gómez-Lázaro, Using SCADA data for wind turbine condition monitoring: A systematic literature review, Energies 13 (12) (2020) 3132. doi:10.3390/en13123132.

[14] L. Sheng, C. Li, M. Gao, X. Xi, D. Zhou, A review of SCADA-based condition monitoring for wind turbines via artificial neural networks, Neurocomputing 633 (2025) 129830. doi:10.1016/j.neucom.2025.129830.

[15] W. Yang, R. Court, J. Jiang, Wind turbine condition monitoring by the approach of SCADA data analysis, Renewable Energy 53 (2013) 365–376. doi:10.1016/j.renene.2012.11.030.

[16] P. Bangalore, M. Patriksson, Analysis of SCADA data for early fault detection, with application to the maintenance management of wind turbines, Renewable Energy 115 (2018) 521–532. doi:10.1016/j.renene.2017.08.073.

[17] K. Leahy, C. Gallagher, P. O'Donovan, D. T. J. O'Sullivan, Issues with data quality for wind turbine condition monitoring and reliability analyses, Energies 12 (2) (2019) 201. doi:10.3390/en12020201.

[18] S. Letzgus, Change-point detection in wind turbine SCADA data for robust condition monitoring with normal behaviour models, Wind Energy Science 5 (2020) 1375–1397. doi:10.5194/wes-5-1375-2020.

[19] N. Renström, P. Bangalore, E. Highcock, System-wide anomaly detection in wind turbines using deep autoencoders, Renewable Energy 157 (2020) 647–659. doi:10.1016/j.renene.2020.04.148.

[20] I. Vervlimmeren, X. Chesterman, T. Verstraeten, A. Nowé, J. Helsen, Scalable scada-driven failure prediction for offshore wind turbines using autoencoder-based nbm and fleet-median filtering, Wind Energy Science 10 (2025) 2615–2637. URL https://wes.copernicus.org/articles/10/2615/2025/

[21] P. Martín-Calzada, P. Martín Sánchez, F. J. Rodríguez-Sánchez, C. Santos-Pérez, J. Ballesteros, Optimized sensor data preprocessing using parameter-transfer learning for wind turbine power curve modeling, Sensors 25 (17) (2025) 5329. doi:10.3390/s25175329.

[22] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, J. Zhang, Edge intelligence: Paving the last mile of artificial intelligence with edge computing, Proceedings of the IEEE 107 (8) (2019) 1738–1762. doi:10.1109/JPROC.2019.2918951.

[23] H. B. Gooi, Z. Wang, Y. Tang, Edge intelligence for smart grid: A survey of applications, techniques and challenges, CSEE Journal of Power and Energy Systems 9 (5) (2023) 1623–1640. doi:10.17775/CSEEJPES.2022.02210.

[24] A. Mwangi, R. Sahay, E. Fumagalli, M. Gryning, M. Gibescu, Towards a software-defined industrial IoT-edge network for next-generation offshore wind farms: State of the art, resilience, and self-x network and service, Energies 17 (12) (2024) 2897. doi:10.3390/en17122897.

[25] D. Astolfi, F. Castellani, A. Garinei, L. Terzi, Data mining techniques for performance analysis of onshore wind farms, Applied Energy 148 (2015) 220–233. doi:10.1016/j.apenergy.2015.03.075.

[26] Y. Liu, J. Wang, L. Liu, Physics-informed reinforcement learning for probabilistic wind power forecasting under extreme events, Applied Energy 376 (2024) 124068.

[27] W. Liao, J. Fang, L. Ye, B. Bak-Jensen, Z. Yang, F. Porté-Agel, Can we trust explainable artificial intelligence in wind power forecasting?, Applied Energy 376 (2024) 124273. doi:10.1016/j.apenergy.2024.124273.

[28] J. Wang, A. Botterud, R. Bessa, H. Keko, L. Carvalho, D. Issicaba, J. Sumaili, V. Miranda, Wind power forecasting uncertainty and unit commitment, Applied Energy 88 (11) (2011) 4014–4023.

[29] H. Quan, D. Srinivasan, A. M. Khambadkone, A. Khosravi, A computational framework for uncertainty integration in stochastic unit commitment with intermittent renewable energy sources, Applied Energy 152 (2015) 71–82.

[30] D. Larrahondo, R. Moreno, H. R. Chamorro, F. Gonzalez-Longatt, Comparative performance of multi-period acopf and multi-period dcopf under high integration of wind power, Energies 14 (15) (2021) 4540. doi:10.3390/en14154540.

[31] T. Burton, N. Jenkins, D. Sharpe, E. Bossanyi, Wind Energy Handbook, 2nd Edition, Wiley, 2011.

[32] International Electrotechnical Commission, IEC 61400-12-1:2017 wind energy generation systems – part 12-1: Power performance measurements of electricity producing wind turbines, International Standard, edition 2.0 (2017).
URL https://webstore.iec.ch/en/publication/26603

[33] S. M. Parsa, Physics-informed machine learning meets renewable energy systems: A review of advances, challenges, guidelines, and future outlooks, Applied Energy 402 (2025) 126925.

[34] Y. Wang, F. Zhang, H. Kou, R. Zou, Q. Hu, J. Wang, D. Srinivasan, A review of predictive uncertainty modeling techniques and evaluation metrics in probabilistic wind speed and

wind power forecasting, Applied Energy 396 (2025) 126234. doi:10.1016/j.apenergy.2025.126234.

[35] J. Liu, L. Wang, T. Jiao, W. Yang, R. Xu, K. Wu, H. Ha, Multi-period voltage stability-constrained optimal power flow with uncertainties, Applied Energy 369 (2024) 123522. doi:10.1016/j.apenergy.2024.123522.

[36] K. Li, Y. Mu, F. Yang, H. Wang, Y. Yan, C. Zhang, A novel short-term multi-energy load forecasting method for integrated energy system based on feature separation-fusion technology and improved cnn, Applied Energy 351 (2023) 121823. doi:10.1016/j.apenergy.2023.121823.

[37] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, G. E. Hinton, Adaptive mixtures of local experts, Neural Computation 3 (1) (1991) 79–87. doi:10.1162/neco.1991.3.1.79.

[38] W. Fedus, B. Zoph, N. Shazeer, Switch transformers: Scaling to trillion parameter models, Journal of Machine Learning Research 23 (2022) 1–39.

[39] K. Dragomiretskiy, D. Zosso, Variational mode decomposition, IEEE Transactions on Signal Processing 62 (3) (2014) 531–544. doi:10.1109/TSP.2013.2288675.

[40] Z. Wu, N. E. Huang, Ensemble empirical mode decomposition: A noise-assisted data analysis method, Advances in Adaptive Data Analysis 1 (1) (2009) 1–41. doi:10.1142/S1793536909000047.

[41] R. Schwartz, J. Dodge, N. A. Smith, O. Etzioni, Green AI, Communications of the ACM 63 (12) (2020) 54–63. doi:10.1145/3381831.
URL https://doi.org/10.1145/3381831

[42] E. Strubell, A. Ganesh, A. McCallum, Energy and policy considerations for deep learning in NLP, in: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, 2019, pp. 3645–3650. doi:10.18653/v1/P19-1355.
URL https://aclanthology.org/P19-1355/

[43] A. Botterud, Z. Zhou, Use of wind power forecasting in operational decisions, Tech. rep., Argonne National Laboratory, technical report (2011).

[44] National Energy Administration of China, DL/T 5002-2021 Design code for dispatch automation of regional power grids, DL/T 5002-2021, industry standard issued by the National Energy Administration of China; implemented on 2021-10-26 (in Chinese). (2021).

[45] P. Ellinas, I. Karampinis, I. Ventura Nadal, R. Nellikkath, J. Vorwerk, S. Chatzivasileiadis, Physics-informed machine learning for power system dynamics: A framework incorporating trustworthiness, Sustainable Energy, Grids and Networks 43 (2025) 101818. doi:10.1016/j.segan.2025.101818.

[46] J. Tautz-Weinert, S. J. Watson, Using SCADA data for wind turbine condition monitoring: A review, IET Renewable Power Generation 11 (4) (2017) 382–394. doi:10.1049/iet-rpg.2016.0248.

[47] J. Ren, W. Guo, Z. Liu, M. Wu, X. Du, A review of scada systems in power grids: Security, architecture, and perspectives, IEEE Internet of Things Journal 8 (16) (2021) 12741–12759. doi:10.1109/JIOT.2020.3044893.

[48] C. Draxl, A. Clifton, B.-M. Hodge, J. McCaa, The wind integration national dataset (wind) toolkit, Applied Energy 151 (2015) 355–366. doi:10.1016/j.apenergy.2015.03.121.

[49] S. De Vito, E. Massera, M. Piga, L. Martinotto, G. Di Francia, On field calibration of an electronic nose for benzene estimation in an urban pollution monitoring scenario, Sensors and Actuators B: Chemical 129 (2) (2008) 750–757. doi:10.1016/j.snb.2007.09.060.

[50] T. Hong, P. Pinson, S. Fan, H. Zareipour, A. Troccoli, R. J. Hyndman, Global energy forecasting competition 2012, International Journal of Forecasting 30 (2) (2014) 357–363. doi:10.1016/j.ijforecast.2013.07.001.

[51] F. X. Diebold, R. S. Mariano, Comparing predictive accuracy, Journal of Business & Economic Statistics 13 (3) (1995) 253–263. doi:10.1080/07350015.1995.10524599.